东南大学

数据结构与算法专题实践

外存数据管理与归并排序

实验报告

Project 1 / Project 2 / Project 3

学生姓名: 刘酝潇

学　　号: 09024211

课　　程: 数据结构与算法专题实践

实验主题: 外存数据管理与归并排序

实现语言: C++11

实验环境: Windows 原生 MSVC / WSL2 GCC

2026 年 11 月

本报告采用自主排版，不代表学校官方模板。

摘要

本实验围绕有限工作内存条件下的整数外排序，完成从二进制文件组织到多路归并的连续工程实践。文件采用固定大小文件头、预留目录和顺序数据区，多个不等长有序段共存于同一文件。输入与输出缓冲将逐元素比较转换为分块读写，使算法不必持有完整数据。

Project 1 以固定容量分块内排生成初始段，建立普通二叉归并计划并后序执行。Project 2 用真正保存内部败者的败者树实现置换选择，通过冻结与解冻生成长段；输入、计算和输出采用双缓冲与三个线程的阶段同步，并以哈夫曼方法选择二路归并顺序。Project 3 复用置换选择，在归并阶段维护共享的 $2k$ 输入块、每路队列和空闲池，按最后已读关键字调度预取，完成真正的 K 路选择和多轮归并。

最终代码在 Windows 原生 Debug 和 Release 环境完成完整验收，测试包含 220 次与 `std::sort` 完整结果的精确比较、机制与非法输入检查以及 6 项 CTest。真实实验中，$N = 50003, M = 256, B = 64, k = 4$ 时，初始段由 196 减为 99，Project 3 的数据块读写调用总数为 7970，少于 Project 1 的 13964；但在本次小块、可缓存的数据规模下，阶段线程创建和同步使并行方案耗时更高。报告区分理论搬移代价、应用层块调用和实际计时，说明这些结果的条件与边界。

本文按问题需求、系统结构、模块设计、测试分析、实验总结和完整源码六部分组织。运行截图来自 Windows 实际程序，性能图由保存的 CSV 生成；最后附全部 17 个正式 C++ 文件及构建配置，源码快照与已验收提交对应。

关键词：外存排序；归并段；败者树；置换选择；动态缓冲；阶段同步

目录

摘要 **I**

第一章 问题描述与需求分析 **1**

1.1 问题背景与数据约束 1

1.2 课程要求及实现范围 2

1.3 Project 1：基于计划树的二路外排序 2

1.4 Project 2：长段生成与置换选择 2

1.5 Project 3：K 路归并与动态缓冲调度 3

第二章 系统结构与算法思想 **4**

2.1 统一工程与分层关系 4

2.2 三个项目的数据流与复用 4

2.3 主要源文件职责 5

2.4 关键不变量与错误传播 6

2.5 Git 演进与可复现交付 6

第三章 功能模块设计 **7**

3.1 二进制文件组织 7

3.1.1 物理布局与定位公式 7

3.1.2 Scan、Put、Search 与流式提交 7

3.2 输入与输出缓冲区 8

3.3 固定容量初始归并段生成 8

3.4 二路归并与二叉计划执行 9

3.4.1 节点表示与普通计划 9

3.4.2 后序执行与两路选择 9

3.5 败者树的结构与更新规则 10

3.6 置换选择与冻结机制 10

3.6.1 从固定候选到长归并段 10

3.6.2 段边界、暂时缺块与真正结束 11

3.7 三线程分阶段并行与双缓冲 12

3.7.1 阶段内的数据所有权 12

3.7.2 交接顺序与异常收尾 . . . 12
3.8 哈夫曼最优归并计划 . . . 13
3.9 真正 K 路选择与多轮归并 . . . 13
3.9.1 单组归并的阶段主循环 . . . 13
3.9.2 多轮主控与不足 K 路的末组 . . . 14
3.10 动态输入缓冲池与预取策略 . . . 14
3.10.1 物理块与逻辑队列分离 . . . 14
3.10.2 预取选择与真实跨路回收 . . . 15
3.11 复杂度、内存与 I/O 成本 . . . 15

第四章 测试结果与分析 17
4.1 测试环境与版本 . . . 17
4.2 测试数据与复现方法 . . . 17
4.3 正确性判据与实际结果 . . . 18
4.4 Windows 真实运行截图 . . . 19
4.4.1 Week 3 文件与缓冲验证 . . . 20
4.4.2 Project 1 初始段与计划执行 . . . 20
4.4.3 Project 2 冻结、解冻与最优计划 . . . 21
4.4.4 Project 3 动态池与 K 路归并 . . . 21
4.4.5 全部自动测试与实际断点 . . . 22
4.5 性能指标与 CSV 结果 . . . 22
4.5.1 统计口径 . . . 22
4.5.2 初始段、归并次数与轮数 . . . 23
4.5.3 数据块读写与实际时间 . . . 24
4.6 Windows 与 WSL 的独立比较及解释边界 . . . 25

第五章 实验总结 26
5.1 完成情况与证据链 . . . 26
5.2 实际遇到的问题与解决过程 . . . 26
5.2.1 构建环境与编码差异 . . . 26
5.2.2 测试目录生命周期与文件系统开销 . . . 26
5.2.3 Ctrl+F5 复用终端的工作目录 . . . 27
5.2.4 接口连接、线程收尾和验证口径 . . . 27
5.3 学习收获与验收理解 . . . 27
5.4 局限性与改进方向 . . . 28

5.5 材料来源与复现文件 28

第六章 完整源代码 **29**

6.1 main.cpp 29
6.2 src/buffer.cpp 32
6.3 src/buffer.h 33
6.4 src/kway_merge.cpp 33
6.5 src/kway_merge.h 38
6.6 src/loser_tree.cpp 39
6.7 src/loser_tree.h 40
6.8 src/merge_plan.cpp 41
6.9 src/merge_plan.h 44
6.10 src/projects.cpp 44
6.11 src/projects.h 46
6.12 src/replacement.cpp 46
6.13 src/replacement.h 49
6.14 src/run_file.cpp 50
6.15 src/run_file.h 53
6.16 tests/test_sort.cpp 54
6.17 tests/test_sort.h 60
6.18 CMakeLists.txt 60

第一章　问题描述与需求分析

1.1　问题背景与数据约束

排序算法必须在内存中比较关键字，但实际文件可能远大于可用于排序的工作区。若把全部输入装入 vector 再调用内排序，空间需求随 N 增长，无法说明外存管理机制。本实验把原始数据保存在文件中，排序过程中每次只处理限定数量的整数，再把阶段结果写回文件。

外存算法的代价不仅是比较次数。逐个整数调用文件接口会产生大量细碎访问；将相邻值按块传输可以利用顺序访问，把一次 I/O 服务于多次比较。缓冲区因此承担两个不同粒度之间的衔接：文件按块读写，计算按元素取值或输出。操作系统仍可能缓存这些调用，故程序计数不等于硬盘实际寻道或物理读写次数。

归并段是文件中一个连续、内部非降序的整数序列。多个段分别有序，并不意味着按段号拼接后整体有序。算法先生成段，再反复合并，直到只剩一个逻辑最终段。实现保留旧段以便核验，因此“只剩一个”指当前参与归并的段号列表只剩一个，而非磁盘目录只剩一项。

表 1.1: 全篇符号与单位

符号	含义
N	原始输入的整数总数，每个元素为 `int32_t`。
M	分块内排或置换选择候选容量，以元素个数计。
B	一个数据块可容纳的整数数，常见平台字节容量为 $4B$。
k	每组 K 路归并允许的最多路数，至少为 2。
R	生成阶段产生的初始段数；L_i 表示第 i 段长度。
C	二叉计划中所有内部节点长度之和，即理论合并输出元素量。
S	分阶段并行任务的同步阶段数量，不等同于归并轮数。

如表 1.1 所示，M 不是整个进程的字节内存上限。缓冲区、败者树状态、目录、计划以及测试参考数组另占空间。正式排序入口 `sortFile` 接收文件路径，不接收完整数据数组；测试层为获得独立参考答案，可以持有完整输入及 `std::sort` 结果。二者的职责必须分清。

1.2 课程要求及实现范围

本报告以用户提供的《外存数据管理》课件、两份课堂语音转写和最终任务说明为课程依据。课件第 75–81 页给出 Week 3–8 推进顺序，分别覆盖文件与缓冲、归并计划、置换选择、动态缓冲、多路归并及测试评估。课堂说明要求最终报告附带有注释的源码，并能够解释重要函数和数据结构。

三项项目在同一仓库中逐步演进。矩阵文件示例用于理解 Header、Directory 和 Data 的组织思想；本实现将对象换为一维归并段。矩阵乘法、快排外排序、多块缓存替换和脏块机制不属于本次实现范围。这样的取舍使每一层都有明确输入输出，课堂演示可以直接追踪整数在文件、块和候选树之间的移动。

1.3 Project 1：基于计划树的二路外排序

Project 1 首先解决可运行的完整外排序流程。输入是无文件头的连续本机字节序整数；输出是带目录的归并文件，以及说明最终段位置的 `finalRun`。生成阶段每次读取不超过 M 个值，在内存中排序后按 B 写成一个有序段。由此得到至多 $\lceil N/M \rceil$ 个初始段。

归并前先建立二叉计划。叶子引用已经存在的初始段，内部节点记录一次未来归并及两个孩子。计划构造与执行分开：先能打印整棵树、核对长度，再按后序递归执行。两路归并只保留两个输入块和一个输出块，不把两个完整段同时装入内存。奇数个节点时，最后一项带入下一层，避免无意义复制。

关键验收目标是：文件偏移正确、每个初始段及中间段有序、递归依赖正确、尾块不丢失、最终内容与参考排序逐项相同。仅显示“输出有序”不足以排除丢失或重复元素。

1.4 Project 2：长段生成与置换选择

普通分块生成的段长最多为 M。若候选集合持续接纳仍能排在当前输出后面的新数，就有机会生成更长段，降低后续归并工作。Project 2 维护固定 M 个候选，用败者树选出当前最小值，每输出一个再补入一个。若补入值小于刚输出值，则冻结，留给下一段；当前有效候选全部冻结时换段并解冻。

长段长度不等，因此归并顺序有实际影响。采用每次合并最短两段的哈夫曼计划，优化加权元素搬移量。并行发生于置换选择：输入线程预读备用块，计算线程操作活动块和树，输出线程写上一阶段的输出块。每阶段用 `join` 等待全部任务完成，再交接角色；后续二路归并仍为串行。

本项目必须体现真实败者数组、冠军路径更新、冻结状态、双输入/双输出以及段边界处理。不能以对全体数据排序替代置换选择，也不能仅启动空线程来声称完成并行。

1.5 Project 3：K 路归并与动态缓冲调度

Project 3 复用置换选择的初始段，归并阶段同时比较多路头值，使每轮段数按约 k 倍缩减。输入块不能永久固定为每路两个：各路消耗速度取决于关键字大小，某一路可能持续获胜，另一条路暂时不消耗。共享 $2k$ 个物理块、各路队列和空闲池使空闲容量能够转给更需要的路。

预取依据是每路“最后一个已读关键字”，并在仍有未读数据的路中选择其最小者；缺少头值的空队列先补齐。未知头值意味着无法证明其他路当前值最小，因此必须在同步点等待补块。输出仍是一路，使用两块交替。多轮主控要正确处理不足 k 的末组、单段直接带过、空输入以及最终输出排空。

表 1.2: 功能需求与验收依据

需求	实现约束	可核验结果
文件组织	固定头、预留目录、流式追加	重开后偏移及数量正确，Scan/Put/Search 有效
块缓冲	逐元素接口、尾块显式刷新	10 个值、$B = 3$ 时各 4 次数据块读写
二路计划	先建树、后序执行	15 值示例 4 段、3 次归并、$C = 30$
置换选择	M 候选、冻结、败者路径重赛	15 值示例段长 7/7/1
最优计划	每次合并最短两段	7/7/1 的代价由 29 减为 23
阶段线程	三任务独占数据、join 交接	输入/计算/输出有真实任务，最后一块排空
动态 K 路	$2k$ 共享池、每路队列、跨路回收	测试中一路可占 4 块，回收块供另一条路使用
正确性	完整内容与参考排序一致	220 次精确比较及机制断言通过
可演示性	原生构建、具名启动项、断点	一键十项验收与 VS Code 实测

第二章　系统结构与算法思想

2.1　统一工程与分层关系

工程只有一个 `main.cpp` 和一个可执行程序。命令行负责模式分派与验证，`sortFile` 负责连接算法，文件层负责存储；败者树只处理候选，不依赖磁盘。图 2.1 概括这些调用关系。这种分层让三个项目共享底层机制，也便于独立检查某一层的错误。

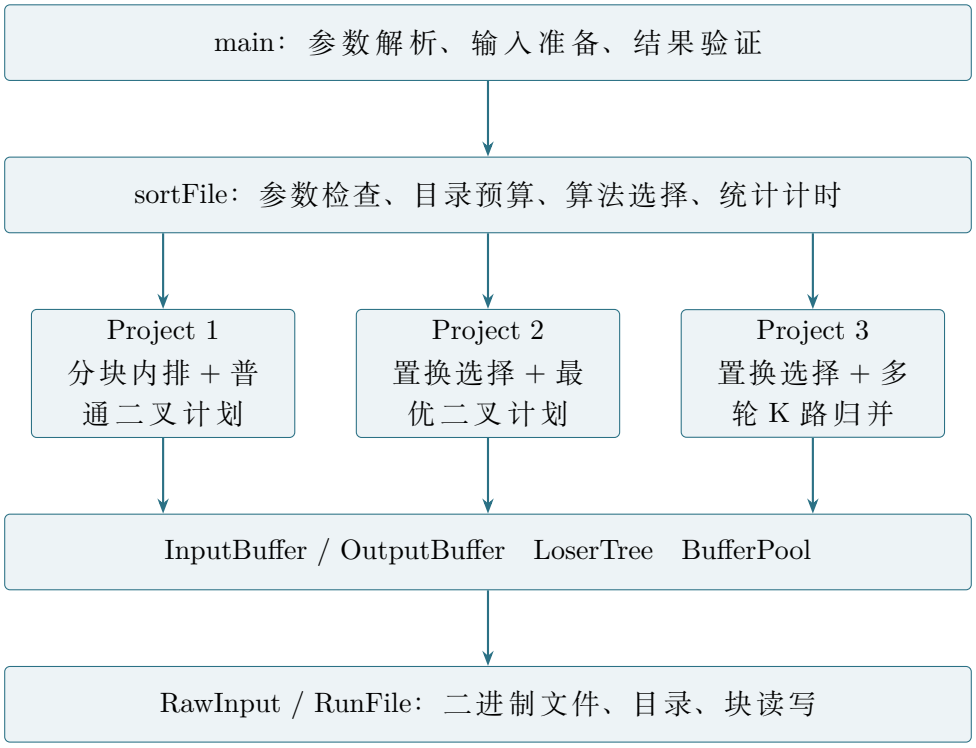


图 2.1: 系统结构与公共模块，箭头表示上层调用下层

`RunFile` 管理已经提交的目录和当前正在追加的段。算法返回目录下标，由调用者继续定位数据；这避免用“返回整个排序数组”的接口破坏外排序的空间约束。所有初始与中间结果顺序追加到同一个运行文件。目录中的旧段保持可读，当前归并列表决定哪些段参加下一轮。

2.2　三个项目的数据流与复用

表 2.1 体现了项目的递进关系。Project 2 替换生成策略和计划策略，但沿用文件、缓冲及二路执行器；Project 3 沿用长段生成与败者树，替换归并主控和输入块管理。K 路阶段直接管理物理块队列，没有再为每一路永久配置两个 `InputBuffer` 对象。

以 Project 2 默认输入为例，`main` 先写 15 个整数；`sortFile` 计算目录容量为 $2\lceil 15/4\rceil + 1 = 9$；数据起点为 $32 + 9 \times 48 = 464$ 字节。置换选择生成长度 7、7、1 的段，最优计划

表 2.1: 三条完整数据处理路线

环节	Project 1	Project 2	Project 3
共同输入	原始本机字节序 `int32_t` 文件，经 RawInput 按块访问		
生成	固定 M 内排	败者树置换选择	复用置换选择
初始结果	RunFile 中命名为 R0、R1 等的有序段		
计划/调度	相邻配对二叉树	最短两段优先二叉树	每轮按至多 k 段分组
执行	后序 + twoWayMerge	复用后序 + twoWayMerge	kWayMerge + BufferPool
返回	新增最终段的目录下标 finalRun，以及 SortResult 统计		

先归并 1 与 7，再归并 8 与 7。最终 `main` 根据返回段号分块读取 15 个值，与独立参考排序逐个核对。

2.3 主要源文件职责

表 2.2: 模块、接口与调用关系

文件组	职责及连接方式
`main.cpp`	number 严格解析数值，usage 说明模式，main 调用演示/测试或 sortFile，再验证输出。
`run_file.h/.cpp`	FileHeader、RunInfo、IoStats、RunFile、RawInput；负责创建文件、目录查询、段追加及数据块访问。
`buffer.h/.cpp`	InputBuffer 提供 next；OutputBuffer 提供 writeValue 和 flush；连接逐元素算法与分块文件接口。
`merge_plan.h/.cpp`	runGeneration、MergeNode、MergePlan、两类建树、twoWayMerge 及 executeMergePlan。
`loser_tree.h/.cpp`	Candidate 的 valid/frozen/value 状态；建树、取冠军、冠军路径重赛、解冻。
`replacement.h/.cpp`	双输入与双输出、输出值的段号标签、三线程置换选择及跨段写入。
`kway_merge.h/.cpp`	InputBlock、BufferPool；预取选择、块生命周期、单组 K 路归并及多轮主控。
`projects.h/.cpp`	SortResult 与 sortFile，连接生成和归并策略，记录计时与计数。

文件组	职责及连接方式
`tests/test_sort.h/.cpp`	机制断言、Week 3 演示、220 次精确对照、固定种子 12 行 Benchmark。
`CMakeLists.txt`	C++11、线程库、警告选项、目标源文件以及 6 项 CTest；不参与运行算法。

每组头文件定义类型及接口，`.cpp` 给出实现。第六章列出全部 17 个正式 C++ 文件，表 2.2 按模块合并展示，不表示省略对应头文件或实现。Windows 的一键脚本、CMake 预设和 VS Code 配置属于验收支持层，不改变排序的模块边界。

2.4 关键不变量与错误传播

文件层保证已提交目录项描述连续且在文件范围内的数据；未完成段只存在于 `pending_`，直到 `endRun` 更新目录。缓冲层保证 $0 \le pos \le count \le B$，只有有效范围被消费。候选树保证所有有效未冻结值优先于冻结值及无效叶，冠军始终代表当前允许输出的最小值。

并行层强调阶段内所有权：输入线程写预留块，计算线程操作已公开块，输出线程只读上一阶段输出。三个任务的异常分别存入 `errors[3]`，主线程全部 `join` 后才读取并重抛。若后续线程创建失败，也先回收已启动线程，再向顶层报告，避免可连接线程在析构时终止进程。

2.5 Git 演进与可复现交付

仓库 https://github.com/David-JayTao/Experiment-External-Memory-Sorting保存一个持续演进工程。`week3-complete` 至 `week8-complete` 对应功能检查点：文件缓冲、二路计划、长段、动态池、K 路主控、最终测试。标签保留当时源码和支持的命令，不将当前所有功能倒写到早期版本，也不改动真实提交时间。

Windows 配置与中文注释分别形成独立中文提交。源码附录从最后验收版本自动生成，版本号写在第六章。个人姓名、学号及正式报告保存在本地被忽略的 `report/` 目录；公开仓库只保留源码及通用验收资料。复现报告只需该目录中的 LaTeX、截图、CSV、矢量图和代码快照，不需要编译产物或安装包。

第三章　功能模块设计

3.1　二进制文件组织

3.1.1　物理布局与定位公式

`FileHeader` 用 32 字节保存标识、目录容量、已提交段数以及目录和数据起点。`RunInfo` 用 48 字节保存 32 字节名称、64 位字节偏移与 64 位元素数量。编译期 `static_assert` 检查结构大小；文件直接保存这些结构的本机表示，不声称能跨字节序交换。

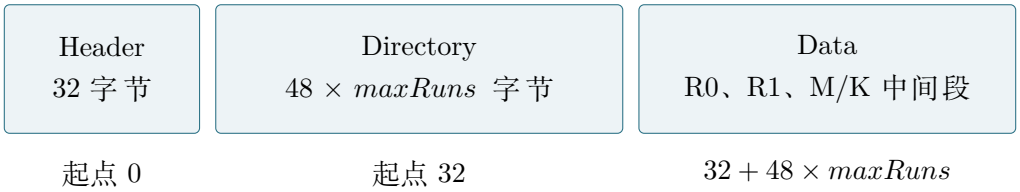


图 3.1: 归并文件物理布局；目录预留区大小在创建时固定

图 3.1 中目录容量与已用数量分开。预留目录保证后续追加一项不会挤占原数据区。第 i 段内零起始下标 p 的整数位于

$$\text{address}(i,p) = \text{RunInfo}[i].offset + p \cdot \text{sizeof}(int32_t), \quad 0 \le p < count_i. \tag{3.1}$$

段号、段内元素下标和文件字节偏移是三个不同概念。`readBlock` 先检查段号，再计算实际数量 $\min(B, count_i - p)$，定位并读取；$p = count_i$ 返回 0，$p > count_i$ 拒绝。每次读写都重新 `seekg/seekp`，不依赖上一次操作留下的流位置。

3.1.2　Scan、Put、Search 与流式提交

`scan()` 返回已加载的目录常量引用，遍历目录不会自动加载段数据；`search(name)` 逐项比较名称，找到返回目录下标，否则返回 -1。`put(name,values)` 是小数据的便利包装，适合 Week 3 与机制测试。正式长段采用表 3.1 中的流式接口。

数据先写、目录后提交，使读取者只通过已完成目录访问完整段。`endRun` 显式刷新并检查流状态，随后另开只读文件可以看到结果；但它不是掉电原子事务。打开文件时检查标识、目录边界、段连续性和数据范围，能发现常见截断或损坏，不等于完整的恶意输入安全解析器。

Week 3 创建容量 8 的目录，数据从 416 字节开始。A 为 $[-5, 2, 8]$，占 12 字节；B 为 $[0, 1]$，起点 428；C 为 $[3, 4, 5, 6, 7]$，起点 436。重新打开后逐块输出仍应相同，Search C 得到 2，missing 得到 -1。这一例子同时检查物理偏移与目录语义。

表 3.1: 一次归并段追加的状态变化

调用	前置条件与行为	状态结果
beginRun	可写、无待写段、目录未满、名称合法且不重复	pending 记录名称与 endOffset，writing 为 true
writeBlock	已开启段，追加 count 个整数；空块跳过	pending.count 加 count，endOffset 加 $4count$
endRun	调用方先刷新尾块，再写目录与头部并 flush	更新 runCount、加入 runs，返回新目录下标

3.2 输入与输出缓冲区

`InputBuffer` 绑定一个段和一个容量为 B 的数组，维护 `nextOffset_`、`pos_`、`count_`。`next` 在 $pos = count$ 时补下一块，推进段内下一读取位置并把 pos 清零；读到 0 说明段耗尽，否则把一个整数通过引用参数交给调用者。长度 7、$B = 3$ 时，底层返回数量依次为 3、3、1、0，只有前三次增加读计数。

`OutputBuffer` 维护有效数量，`writeValue` 填一个值，达到 B 立即 `flush`；最后不足 B 的尾块由调用者显式刷新，再提交段。析构不隐式写盘，避免把可能抛出错误的 I/O 藏在资源销毁过程中。连续调用空刷新不会产生额外非空块计数。

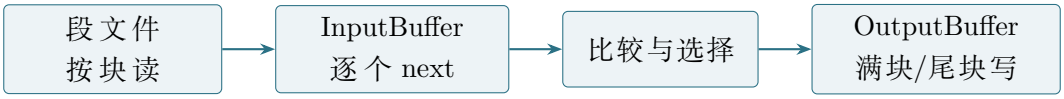


图 3.2: 缓冲区连接块传输与逐元素计算，输出再追加到段文件

图 3.2 中的封装让二路归并只考虑当前值和有效性，而无需在比较循环中重复写补块逻辑。`IoStats*` 可为空，验证层由此不把额外扫描算入排序指标。缓冲对象借用文件引用，文件应在对象使用期间始终存活。

3.3 固定容量初始归并段生成

`runGeneration(rawPath,file,M,B,stats)` 用一个长度 M 的数组。每轮按不超过 B 的块读取，直到凑满 M 或文件结束；只对有效范围调用 `std::sort`，随后用输出缓冲逐个写出，刷新并提交为 `R0`、`R1` 等。返回的是段号序列，不是数据副本。

1. 令当前有效数为 0，按 $\min(B, M - count)$ 读取到工作数组剩余位置。
2. 若实际读入总数为 0 则结束；否则在 $[0, count)$ 内排序。
3. 开启新段，按 B 写完已排序值，尾块刷新后提交目录。
4. 重复直至原始输入耗尽；空输入不生成空初始段。

默认输入 $[9, 1, 8, 2, 7, 3, 6, 4, 5, 0, -1, 3, 12, 2, 10]$ 取 $M = 4$ 得到表 3.2。每个长度 4 的

段在 $B = 3$ 时分两次写，不能把段数或段长度误认为 I/O 块数。

表 3.2: Project 1 默认输入的初始段

段	排序后内容	元素数	字节起点
R0	1, 2, 8, 9	4	464
R1	3, 4, 6, 7	4	480
R2	−1, 0, 3, 5	4	496
R3	2, 10, 12	3	512

比较成本为 $O(N \log M)$，工作数组及输出块为 $O(M + B)$，另有 $O(R)$ 段号及目录。若 M 不整除 B，每个工作区末次读取可能不足块；真实调用数要逐段取整，不能直接等同于 $\lceil N/B \rceil$。

3.4 二路归并与二叉计划执行

3.4.1 节点表示与普通计划

`MergeNode` 包含左右孩子下标、实际段号与长度。叶子孩子为 −1，段号已知；内部节点的段号初始为 −1，执行后记录结果。`MergePlan` 保存节点数组、根和代价。建树先放叶子，再逐层相邻配对，所以孩子下标小于父节点，根在末尾。

图 3.3 是表 3.2 对应的真实计划，方框中表示节点下标与长度。节点 4 先合并 0 与 1，节点 5 合并 2 与 3，最后根 6 合并 4 与 5。建树仅访问目录，真正的数据 I/O 发生于后序执行。

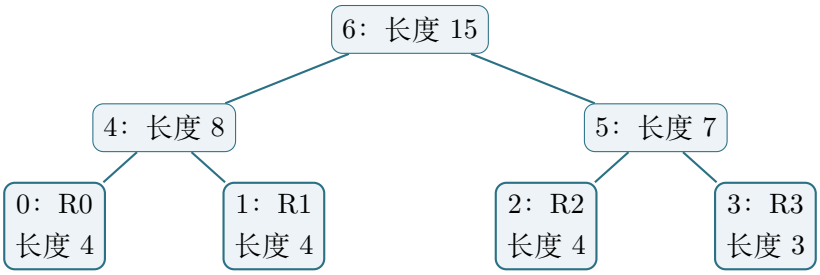


图 3.3: 普通归并计划的具体节点与依赖，总代价 $8 + 7 + 15 = 30$

3.4.2 后序执行与两路选择

`executeMergePlan` 遇到根 −1 返回 −1；遇到已有段号的节点直接返回；否则先递归完成左、右子树，将返回的两个段号交给 `twoWayMerge`，把新段号写回当前节点。缓存结果保证同一计划节点不会被重复执行。递归栈只保存少量状态，不保存各层完整数据。

`twoWayMerge` 建立左右输入缓冲和一个输出缓冲，用 `hasLeft/hasRight` 区分合法值与结束。每次输出较小头值，只推进对应输入；相等优先左路，一方耗尽后排空另一方。

结果仍以新段追加到同一文件。这里使用同一个 `fstream` 是安全的，因为两路读取和写入都在同一线程顺序执行。

若左右长度为 a, b，CPU 比较/移动为 $O(a+b)$，整数缓冲为 $3B$。一次归并数据块调用数为

$$Q_{\text{merge}} = \left\lceil \frac{a}{B} \right\rceil + \left\lceil \frac{b}{B} \right\rceil + \left\lceil \frac{a+b}{B} \right\rceil . \tag{3.2}$$

这里不包含文件头与目录读写。正确性可由不变量说明：输出前缀已排序，且其中每个元素不大于两路剩余值；取较小头值保持这一性质，最终数量为 $a+b$。

3.5 败者树的结构与更新规则

败者树叶子是 `Candidate(value,valid,frozen)`。内部数组 `losers_` 保存每场比赛的败者叶下标，总冠军另存 `champion_`。候选数补齐至 2 的幂，多出的叶子标为无效，不占真实候选名额。`valid=false` 代表 EOF 或补齐叶，避免用 `INT_MAX` 哨兵排除真实最大整数。

比较顺序为有效优先、未冻结优先、数值小优先、叶下标小优先。`build` 从子树取得两个冠军，将败者留在当前内部节点，赢家向父亲返回。`winner` 若发现总冠军无效便返回 -1；`top` 只在仍有冠军时作为有效输出使用。

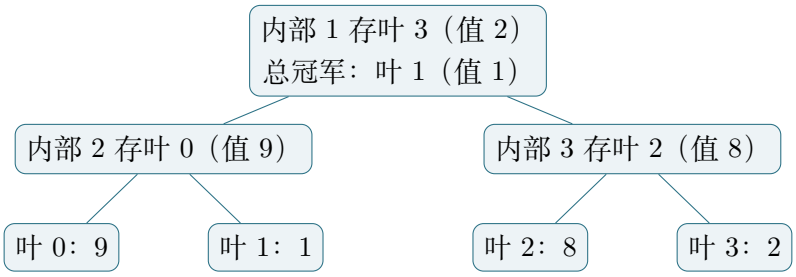


图 3.4: 候选 $[9,1,8,2]$ 的败者树；内部持久存储的是败者下标

图 3.4 中输出 1 后用 7 替换叶 1。7 先与记录的 9 比较获胜，再与记录的 2 比较失败，根留下叶 1，总冠军改为叶 3。只需沿被替换冠军的叶到根路径重赛，因为其他子树的比赛没有变化。该接口不能直接更新任意非冠军叶；本项目两处调用都符合“输出冠军后补入”的条件。

建树与批量解冻重建均为 $O(M)$，单次冠军替换为 $O(\log M)$。在 K 路归并中把 M 换为实际路数。平局按叶下标处理使演示确定，但本项目只排序整数，没有携带记录身份来承诺任意记录的全局稳定性。

3.6 置换选择与冻结机制

3.6.1 从固定候选到长归并段

`replacementSelection` 先按块装入至多 M 个候选，构造败者树，再预载活动输入。

每输出一个冠军，从活动输入补一个新值。如果新值不小于刚输出值，则仍能接在当前非降序段后；若更小，则保留在候选中并冻结。所有有效候选均被冻结时，总冠军也会被冻结，程序增加段号并 `thaw`。

表 3.3: 默认 15 值输入的第一段置换过程，$M = 4$

输出	补入	新候选状态	解释
1	7	活动	$7 \geq 1$
2	3	活动	$3 \geq 2$
3	6	活动	$6 \geq 3$
6	4	冻结	$4 < 6$，不能接入当前段
7	5	冻结	$5 < 7$
8	0	冻结	$0 < 8$
9	-1	冻结	全部有效候选冻结，结束 R0

表 3.3 得到 R0=$[1, 2, 3, 6, 7, 8, 9]$。解冻后生成 R1=$[-1, 0, 3, 4, 5, 10, 12]$，再生成 R2=$[2]$。三个段长为 7、7、1，偏移为 464、492、520。输入中重复值通过严格小于判断保留在当前段，不会因相等而不必要地冻结。

对于升序输入，新数持续满足条件，有可能只生成一个长段；对于逆序输入，新数频繁冻结，不能保证长于 M。随机输入常出现平均接近 $2M$ 的段长，但这依赖分布和模型，本实现不把它作为任意输入的长度保证。

3.6.2 段边界、暂时缺块与真正结束

输出内存块可能跨段，`GeneratedBlock` 因此同时保存 `values` 和逐元素 `runNumbers`。默认 $B = 3$ 时第三块为 $[9, -1, 0]$，段号为 $[0, 1, 1]$。`writeGeneratedBlock` 把它拆成同段连续区间，先写 9 并提交 R0，再开启 R1 写 $[-1, 0]$。一个内存块由此可能对应多次数据写调用。

计算线程区分两种状态：活动输入块暂时耗尽但后面仍有数据，应结束本阶段等待；原始输入真正全部读完，才用无效候选逐个替换并排空树。阶段开始的 `noMoreAfterActive` 是由主线程确定的快照，避免计算线程读取输入线程正在修改的备用块计数。

即使树已空，上一阶段输出仍可能未写盘。循环条件必须同时检查树和待写输出，最后再关闭仍打开的段。这个排空规则与显式尾块刷新共同保证数量不丢失。

3.7 三线程分阶段并行与双缓冲

3.7.1 阶段内的数据所有权

置换选择有两输入块和两输出块。`activeInput` 供计算消费，另一个输入块供预取；`activeOutput` 供计算填充，另一个输出块供写回。两个索引只在主线程完成三个 `join` 之后更新。图 3.5 展示稳定阶段的任务分工。

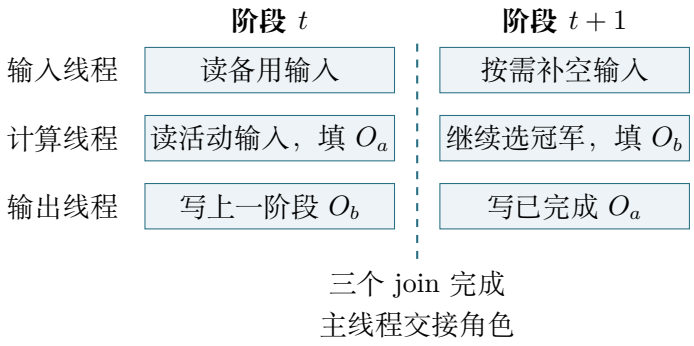


图 3.5: 三线程的阶段同步；输入仅在耗尽时切换，输出每阶段交替

表 3.4: 阶段内可写状态与同步后的处理

执行者	置换选择	K 路归并
输入线程	原始读流、备用输入块、readBlocks	独立只读 source、预留物理块、readBlocks
计算线程	树、活动输入位置、活动输出、currentRun	树、消费队列、released、活动输出、emitted
输出线程	上阶段输出、段文件、writtenRun、writeBlocks	上阶段输出、段文件、writeBlocks
主线程	join 后累加 loaded、按需换输入、换输出	join 后归还块、发布预取块、换输出

表 3.4 说明无需在本实现中引入互斥锁的原因：阶段内可写对象不重叠，主线程不提前观察正在写的数据。输入和输出分别修改 `IoStats` 中不同标量成员；阶段数仅在 join 后更新。日志也限定由计算线程或同步后的主线程输出。

3.7.2 交接顺序与异常收尾

活动输入不一定在每个阶段都恰好用完，例如输出已满时还留有未消费输入。程序只有在 $pos = count$ 时清空该块并切换，避免无条件换块跳过数据。输出线程处理完旧块后，主线程清空它，作为下一阶段的计算输出。

每阶段新建三个 `std::thread`，随后逐个 join。它们可以并行执行，但某个阶段可能没有预取任务或没有上一块可写，启动和排空阶段尤其如此。这个实现服务于课程的阶

段同步机制，不是持续运行的线程池。阶段总耗时受最慢任务、线程创建以及调度影响，不能简单把单线程时间除以三。

线程内部异常必须捕获到各自的 `exception_ptr` 槽；主线程 join 之后再重抛。对线程创建失败的路径也保留已启动线程的 join 收尾。测试未故意耗尽系统线程资源，因此报告只声明该分支经代码审查，不将正常测试冒称为已覆盖此类系统故障。

3.8 哈夫曼最优归并计划

`buildOptimalPlan` 将每个段转换为叶子，将 $(length, nodeIndex)$ 放入最小优先队列。每次弹出两个最短节点，建立长度为两者之和的父节点，并重新入队，直到剩下根。等长时用节点下标确定先后，计划可重复。

设初始段 i 经过 d_i 次归并，则每个元素被复制 d_i 次，总代价满足

$$C = \sum_{v\text{为内部节点}} L_v = \sum_{i=1}^{R} L_i d_i. \tag{3.3}$$

哈夫曼方法优化这一加权路径长度。它并不直接优化尾块取整、目录访问、缓存行为或 CPU 时间，因此最优计划不等于实际计时总是最快。

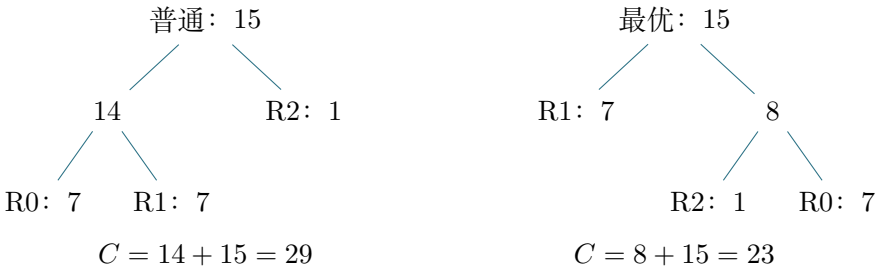


图 3.6: 实际段长 7、7、1 的两种计划，执行器均复用二路后序归并

图 3.6 中最优计划先生成长度 8 的 M3，再生成长度 15 的 M4。独立机制测试另用段长 1、2、3、100 验证普通代价 212、最优代价 115，防止只凭一次演示判断建树正确。建树的 CPU 成本为 $O(R \log R)$，节点和优先队列为 $O(R)$。

3.9 真正 K 路选择与多轮归并

3.9.1 单组归并的阶段主循环

`kWayMerge(file,runs,k,B,stats,trace)` 接收至多 k 个段号，复制本组元数据并求总长度，另开只读 `RunFilesource`，初始化共享池，在原文件开启新 K 段。读线程访问已完成段，写线程追加后续区域，两个线程不共享同一个 `fstream` 对象。

图 3.7 中的计算任务不是多次二路归并。它把实际各路头值组成一棵败者树，每输出一个冠军，消费该路一个值，再用该路新头替换冠军。若某条尚未结束的路暂时没有

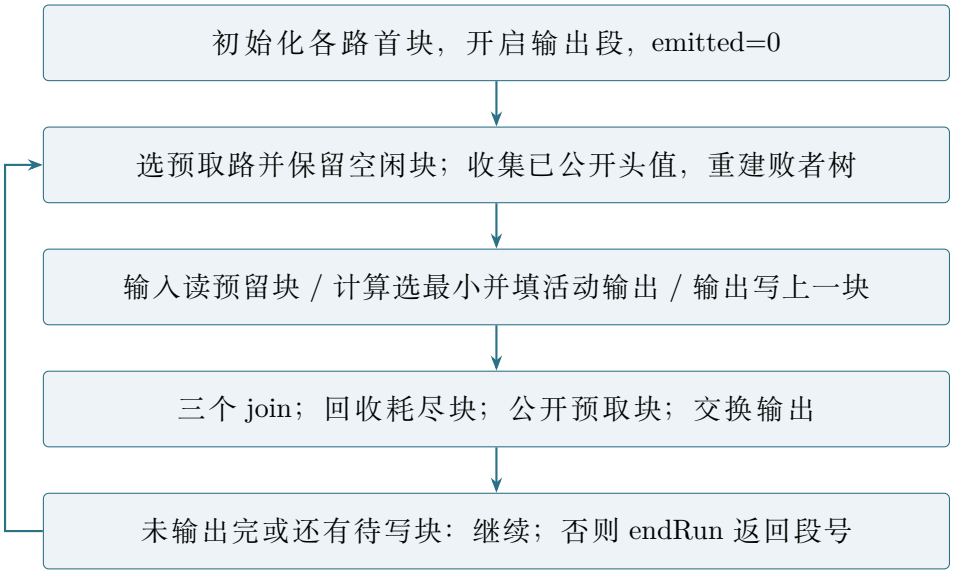


图 3.7: K 路归并阶段流程，循环末尾必须包含写出最后一块

头值，`missingHead` 使本阶段等待补块；若消费过程中出现这种状态，也立即结束本阶段计算。忽略未知头值会破坏全局最小值判断。

实现每阶段重新构造头值树，以纳入同步后公开的新块；阶段内部才使用冠军路径重赛。因此复杂度必须包括每阶段 $O(k)$ 建树和扫描，不能只写理想的 $O(N \log k)$。每阶段最多产生 B 个值，缺头时可能少于 B，尾输出也需完整写出。

3.9.2 多轮主控与不足 K 路的末组

`mergeKRuns` 把当前段号每 k 个分组；组内至少两个时调用一次 K 路归并，单个段直接带入下一轮。一次归并产生一个新段，但磁盘上的旧段保持不变。段号列表替换后继续，直到只剩一个；空输入返回 -1。

例如实际 99 段、$k = 4$ 时，段数为 $99 \to 25 \to 7 \to 2 \to 1$，共 4 轮。归并组数为 $25+6+2+1 = 34$：第二轮 25 段的最后单段直接带过，并不新增一次合并。`rounds` 是多轮主控次数，`merges` 是产生新段的归并组数，`stages` 是线程同步阶段，三者不可互换。

3.10 动态输入缓冲池与预取策略

3.10.1 物理块与逻辑队列分离

`BufferPool.blocks_` 一次分配 $2k$ 个物理块，每块有整数数组、有效数量和消费位置。`queues_[lane]` 只保存块号，`free_` 保存可再分配块，`released_` 暂存本阶段刚耗尽的块。`next_` 记每路已读数量，`lengths_` 是总长度，`lastKeys_` 是所有已读数据的末值。

图 3.8 中物理块和路没有永久绑定。初始化为每个非空段读一块，其余块保留为空闲；下一次预取用 `reserve` 取出一个块。读线程只填该块，join 后 `finishPrefetch` 才把块号入队并更新 `next_` 和 `lastKeys_`。消费块由计算线程出队，但仅登记到 `released_`，

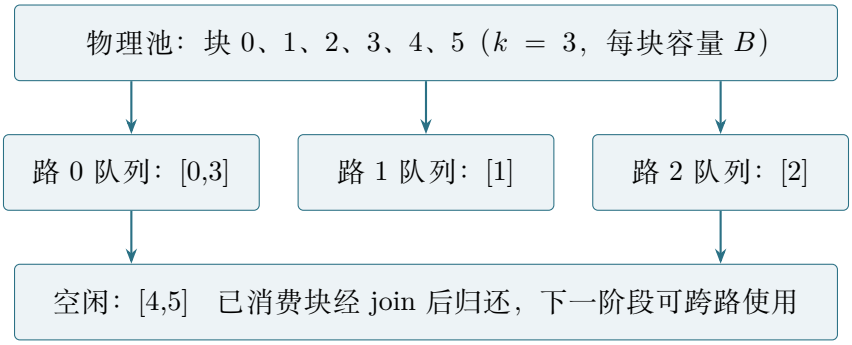


图 3.8: 默认示例第一个 K 路计算阶段同步后的队列状态

主线程同步后统一归还，避免块仍被读取时就被重写。

3.10.2 预取选择与真实跨路回收

正常预取规则可写为

$$\text{target} = \arg \min_{i:\ next_i < length_i} \text{lastKeys}_i, \tag{3.4}$$

并附加“仍有磁盘数据的空队列优先”规则。`lastKeys` 不是队首最小值，也不是尚未读取的整段最大值；它由最近完成读取的块末值更新。已经全部读入的路不再预取，即使其队列尚有很多未消费数据。

默认三路首块分别为 $[1,2,3]$、$[-1,0,3]$、$[2]$。路 2 已全部读入；路 0、1 末值相同，先选路 0 读 $[6,7,8]$，之后选末值仍为 3 的路 1。路 2 的物理块 2 被消费后回收，后续日志出现 `prefetchlane=1block=2`，用来读取路 1 的最后值 12。这是实际跨路复用，不是仅在结构命名上叫“动态池”。

独立测试还让路 0 占有 4 个物理块、其他两路各占 1 块，共 6 块，证明并非永久每路两个。当没有可用块时 `reserve` 返回 -1，本阶段不执行预取；计算继续释放块后再尝试。调度强调正确交接，不对所有系统时序承诺零等待。

3.11 复杂度、内存与 I/O 成本

表 3.5: 核心算法的 CPU 与额外内存成本

模块	CPU 成本与条件	额外内存
固定分块生成	$O(N \log M)$，段数 $\lceil N/M \rceil$	$O(M+B)$ 整数工作区，另有 $O(R)$ 元数据
普通二叉计划	$O(R)$ 建树；后序遍历 $O(R)$，数据合并总计 $O(C)$	$O(R)$ 节点，递归深度通常 $O(\log R)$

模块	CPU 成本与条件	额外内存
哈夫曼计划	$O(R\log R)$ 建树；执行总数据处理 $O(C)$	$O(R)$ 节点和队列，最坏树深 $O(R)$
单次二路归并	长度 a,b 时 $O(a+b)$	$3B$ 个整数及常量状态
败者树	建树/解冻 $O(M)$，一次更新 $O(\log M)$	$O(M)$ 候选和败者下标
置换选择	$O(N\log M+RM)$，另有 S 次阶段调度	$O(M+B)$，含双输入、双输出及段号标签
K 路单组归并	总长度 T 时 $O(T\log k+S_gk)$，S_g 为本组阶段数	$O(kB+k)$，含 $2k$ 输入和 2 输出块
预取及块管理	每阶段选择 $O(k)$；每个块入队、出队、回收均摊 $O(1)$	队列和状态 $O(k)$，块数据计入上项

表 3.5 区分了核心算法与实现附加项。`beginRun` 为检查重名进行线性目录查询，累计创建 D 段可能引入 $O(D^2)$ 名称比较；K 路每次另开只读 RunFile 还会读取当前目录。`sortFile` 在 Project 3 也构造一次普通计划用于 cost 指标。这些元数据工作没有出现在理想归并的比较模型里，但属于实际运行时间。全部目录和历史计划仍需 $O(R)$ 内存，不能宣称整个进程空间恒定。

忽略部分块、元数据及系统缓存，若每轮将全部 N 个值读写一次，初始段数为 R，K 路归并层数约为 $L=\lceil\log_k R\rceil$，理想数据传输约为

$$Q_{\text{ideal}}\approx 2\frac{N}{B}(1+L). \tag{3.5}$$

首个 1 代表生成段的一读一写。真实实现有单段带过、各段尾块、置换输出跨段分拆、K 路缺头提前结束阶段，因此应以 `readBlocks/writeBlocks` 为应用层调用的实测依据。最优树的 C 衡量元素量，也不能未经换算直接称为块 I/O。

内存比较亦有条件：二路归并使用 $3B$ 整数块，K 路使用 $(2k+2)B$，即使各项目设置同样 M,B 也不是同一总字节预算。哈夫曼减少搬移、置换选择减少段数、K 路减少轮数，各自优化不同环节；线程创建与 join、缓存、文件系统会共同决定最终耗时。

第四章　测试结果与分析

4.1　测试环境与版本

实验开发与 Windows 验收于 2026 年 11 月 9–10 日进行，报告整理延续至 11 月 11 日。表 4.1 分别列出 Windows 原生环境与此前 WSL 环境。两者的编译器、文件系统和计时不同，不能将其数据混为同一次实验。

表 4.1: 实际测试环境

项目	Windows 原生	此前 WSL2
操作系统	Windows 11，10.0.26200	Ubuntu 26.04 LTS，内核 6.18.33.2-microsoft-standard-WSL2
CPU	Intel Core i5-13500H，16 个逻辑 CPU	
编译器	MSVC 19.44.35229	GCC/g++ 15.2.0
工具集/SDK	14.44.35207 / Windows SDK 10.0.26100.0	Linux 标准开发与线程库
构建	CMake 4.4.4，C++11，Debug/Release	CMake 4.4.4，C++11，Release 及检查配置
警告选项	/W4 /utf-8，最终无编译警告	-Wall -Wextra -Wpedantic
数据位置	build/windows/run/配置；CTest 在构建目录	Linux 用户缓存目录，非/mnt/c
编辑与调试	VS Code 1.145.2，C/C++ 1.35.6，CMake Tools 1.24.42	命令行构建与测试

微软 Build Tools 安装包经 Microsoft 数字签名验证，在用户确认 UAC 后安装成功，复用现有 CMake，不安装完整 IDE。原生脚本通过 `vswhere` 发现 MSVC，自动配置、增量构建并按真实退出码汇总结果。独立中文及空格路径也已从零编译并运行 Project 3 通过。

注释补充后的最终回归使用同一套十项验收，Debug 和 Release 均通过。对全部 17 个 C++ 文件剥离注释后逐项比较词法单元，与 Windows 兼容修正后的基线一致；没有更改算法、线程同步、文件格式或接口。最终提交及完整源码快照见第六章。

4.2　测试数据与复现方法

默认小输入为 15 个数，$M = 4, B = 3, k = 3$，包含负数和重复值，能在少量输出内展示冻结、尾块、计划树及块回收。最终参考序列为

$$[-1, 0, 1, 2, 2, 3, 3, 4, 5, 6, 7, 8, 9, 10, 12]。$$

自动测试有五组固定边界输入：空、单值、短逆序、重复负数和包含 `INT32_MIN`、`INT32_MAX` 的数组。随后使用 `mt19937(20261109)` 产生 17、257、4099 长度的值，取模 101 再减 50，并派生升序、逆序版本，共 14 组。每组测试 P1、P2 及 P3 的 $k = 2, 3, 4$，合计 70 次。

再对固定 15 值组合 $M = 1, 3, 8$ 和 $B = 1, 5, 17$，三个项目共 27 次；40 个种子 0–39 各产生一组长度小于 100、值域 $[-5, 5]$ 的输入，随机选择 $M = 1 \ldots 9, B = 1 \ldots 11, k = 2 \ldots 7$，共 120 次。最后三个项目各检查 50003 个整数，总计 $70 + 27 + 120 + 3 = 220$ 次。

Benchmark 单独对 $N = 31, 257, 4099, 50003$ 运行三个项目，固定 $M = 256, B = 64, k = 4$。每个规模重新初始化种子 20261109，取模 20001 再减 10000，各项目用相同输入。它的 12 次精确比较不包含在前述 220 次之中。`--size` 演示与 Benchmark 均重新初始化随机生成器；测试套件末尾大输入使用套件内已经推进的生成器状态，二者不应被误认为逐项相同输入。

```
# 在仓库根目录的 PowerShell 中运行
.\验收.ps1 all -Configuration Debug
.\验收.ps1 all -Configuration Release
.\验收.ps1 project3 -K 2
.\验收.ps1 project3 -K 4 -Large
.\验收.ps1 benchmark -Configuration Release
```

同一运行目录不并发启动多个程序，防止固定测试文件名相互覆盖。不同平台采用独立构建目录；输出文件会重建，不能指向原始输入或其路径别名。报告所用 CSV 保留在 `data/`，制图脚本只读取该快照，不把后续重跑时间自动替换进原表。

4.3 正确性判据与实际结果

`checkSort` 独立复制输入并调用 `std::sort`，从最终段分块读回，依次检查数量相等、非降序、逐项相等，再扫描每一个中间段检查有序。逐项相等同时覆盖重复次数；只检查有序性无法发现某些丢值问题。Release 下断言使用抛异常的 `require`，不会因 `NDEBUG` 失效。

表 4.2: 最终 Windows 回归结果与此前 WSL 检查

环境/配置	一键验收或检查	CTest	CTest 总时间
Windows Debug	十项全部 PASS，220 次精确比较，12 行 Benchmark	6/6	32.42 s
Windows Release	十项全部 PASS，220 次精确比较，12 行 Benchmark	6/6	26.85 s
此前 WSL Release	最终功能与 Benchmark 验证	6/6	10.29 s
此前 WSL ASan+UBSan	覆盖运行未见相关诊断	6/6	54.29 s
此前 WSL TSan	覆盖运行未见竞争诊断	6/6	14.02 s

表 4.2 的时间是整个 CTest 套件时间，包含参考排序及所有检查，不是某次排序耗时。WSL 的 Sanitizer 属于此前版本记录，不能写成 Windows 已经运行 Sanitizer，也不是对所有输入的形式化证明。六项 CTest 为 correctness、帮助和四种非法选项；非法参数测试以 `WILL_FAIL` 要求非零退出。

表 4.3: 关键机制的目标、输入和判定

机制	输入/操作	实际断言结果
文件与尾块	10 个数、$B = 3$，重开后读取	4 次非空读、4 次非空写，值完整
目录边界	容量 2，已有两段后再 beginRun	拒绝；重名和无效段号亦拒绝
文件完整性	仅 1 字节裸文件、截断头部	拒绝，未被当作合法整数流
败者树	5 叶候选 4/3/6/8/1，冻结补 0	冻结不抢先、重赛顺序正确、解冻后 0 可输出
最优树	段长 1/2/3/100	普通 212，最优 115
共享池	$k = 3, B = 1$，三路不同值域	一路占 4 块、块 0 回收后供路 1
空路	K 路组内含空段、全空段	分别得到正确长度 1 和 0
CLI 错误	B=0、M=-1、k=1、缺少 size 值	退出非零，CTest 判定通过

4.4 Windows 真实运行截图

以下图像直接采自 VS Code 运行真实 Windows 程序的窗口，LaTeX 只裁去无关界面并按比例排版。截图来自 Windows 验收阶段，后续注释提交未改算法；最终回归记录见表 4.2。截图中 Debug、trace、调试事件及暂停都可能影响计时，不能代替后文的

Release 性能数据。

4.4.1 Week 3 文件与缓冲验证

图 4.1 显示 Header=32、Directory=32、Data=416，A/B/C 三段的实际偏移和长度，以及 Search C=2、missing=−1。C 通过容量 2 的输出缓冲写入 5 个数，包含尾块；重开后扫描值完整，符合模块设计。

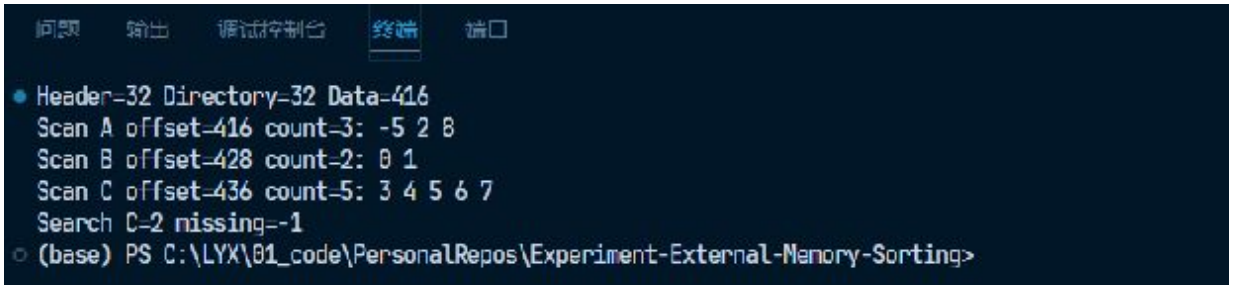


图 4.1: Week 3：真实文件创建、扫描、追加后的查询结果

4.4.2 Project 1 初始段与计划执行

图 4.2 给出四个初始段、节点 0–6 的长度及孩子关系，再显示三次归并和最终 15 值。`theoretical_cost=30` 与手算一致，最终完整比较 PASS。段目录下标 6 对应 M6，文件中仍保留初始与中间段。

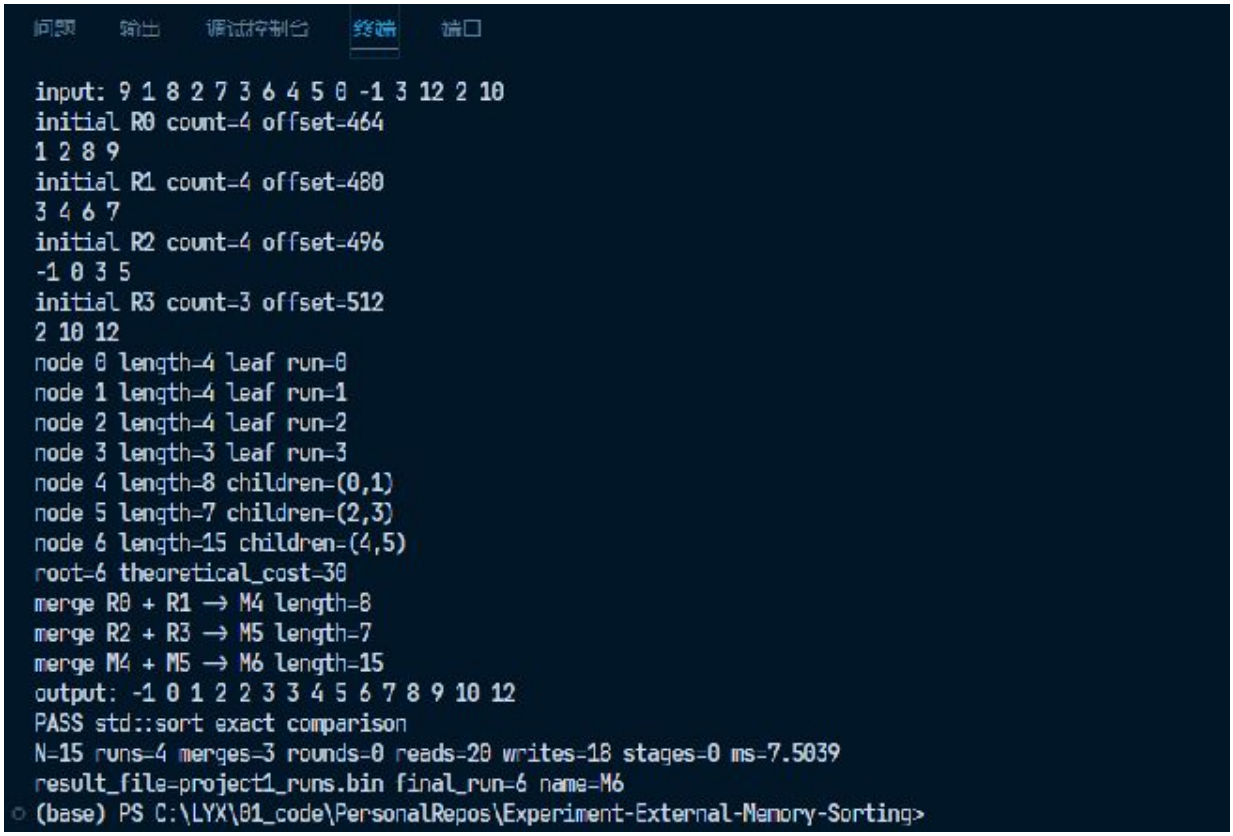


图 4.2: Project 1：分块内排、普通二叉计划及后序执行

4.4.3 Project 2 冻结、解冻与最优计划

图 4.3 可看到候选 2 被标记 frozen，随后 `thaw->run2`；段长为 7/7/1，普通代价 29、最优 23，执行 R2+R0 再与 R1 合并。最终精确结果相同，生成阶段 6 个同步阶段已经包含最后输出排空。

```
champion=3 losers: 1:1 2:0 3:2 leaves: 0=2(frozen) 1=5 2=12 3=4
champion=1 losers: 1:3 2:0 3:2 leaves: 0=2(frozen) 1=5 2=12 3=10
champion=3 losers: 1:0 2:1 3:2 leaves: 0=2(frozen) 2=12 3=10
replacement sync 4 input=1 consumed=2/2 output=1 count=3
champion=2 losers: 1:0 2:1 3:3 leaves: 0=2(frozen) 2=12
champion=0 losers: 1:2 2:1 3:3 leaves: 0=2(frozen)
thaw → run 2
champion=-1 losers: 1:2 2:1 3:3 leaves:
replacement sync 5 input=0 consumed=0/0 output=0 count=3
replacement sync 6 input=1 consumed=0/0 output=1 count=0
initial R0 count=7 offset=464
1 2 3 6 7 8 9
initial R1 count=7 offset=492
-1 0 3 4 5 10 12
initial R2 count=1 offset=520
2
fixed_chunk_runs=4 ordinary_cost=29 optimal_cost=23
node 0 length=7 leaf run=0
node 1 length=7 leaf run=1
node 2 length=1 leaf run=2
node 3 length=8 children=(2,0)
node 4 length=15 children=(1,3)
root=4 theoretical_cost=23
merge R2 + R0 → M3 length=8
merge R1 + M3 → M4 length=15
output: -1 0 1 2 2 3 3 4 5 6 7 8 9 10 12
PASS std::sort exact comparison
N=15 runs=3 merges=2 rounds=0 reads=16 writes=15 stages=6 ms=48.1798
result_file=project2_runs.bin final_run=4 name=M4
(base) PS C:\LYX\01_code\PersonalRepos\Experiment-External-Memory-Sorting>
```

图 4.3: Project 2：冻结状态、解冻、长段及最优二路计划

4.4.4 Project 3 动态池与 K 路归并

图 4.4 展示每路队列中的物理块号、消费位置、loaded、last 以及预取目标。`prefetchlane=1block=2` 证实原来属于路 2 的块回收后服务路 1；末尾 `produced=0` 表示写出最后待写块。三路本例一次归并、一轮完成，最终段为 K3。

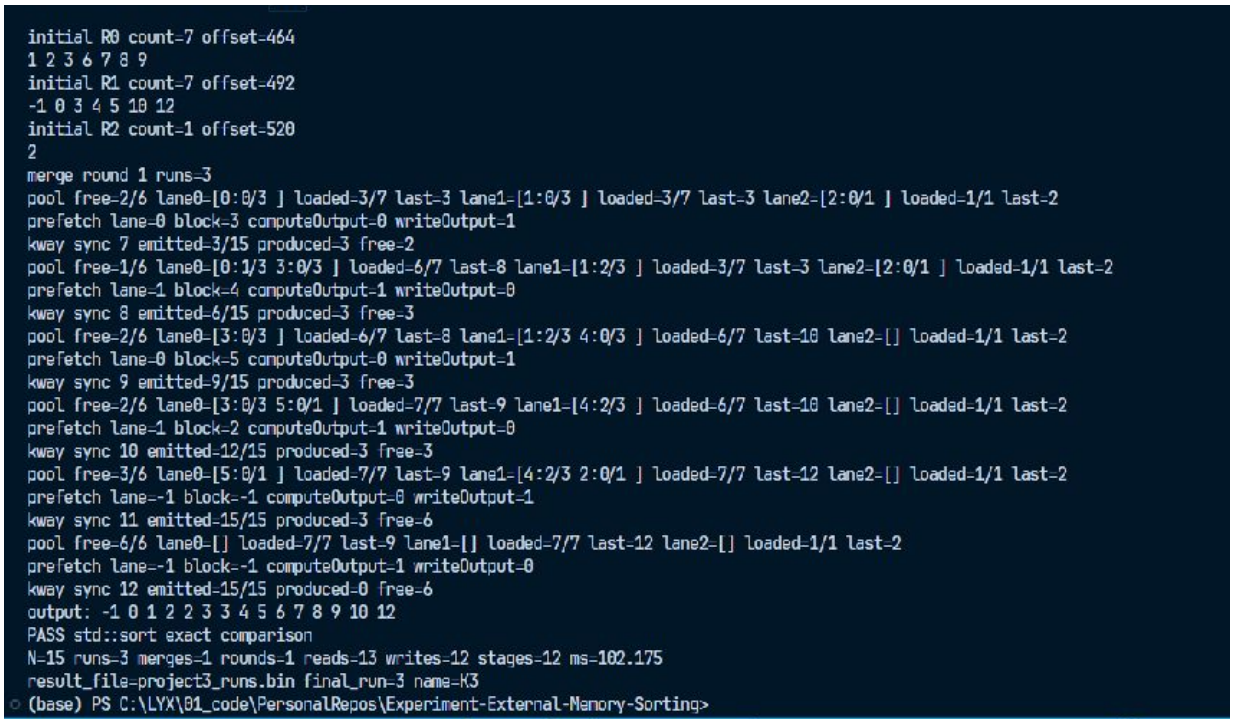


图 4.4: Project 3：动态预取、跨路块复用与最后阶段排空

4.4.5 全部自动测试与实际断点

图 4.5 同时显示机制测试、动态池测试与 220 次完整对照通过。九个具名配置均已实际启动；在 `choosePrefetch` 处 F5 命中断点，F10 后观察到 lane=0，并能查看调用堆栈。临时断点随后清除，继续运行得到 PASS。另一次 Ctrl+F5 验证表明运行目录中的输入文件更新时间改变，仓库根旧文件未被改写。

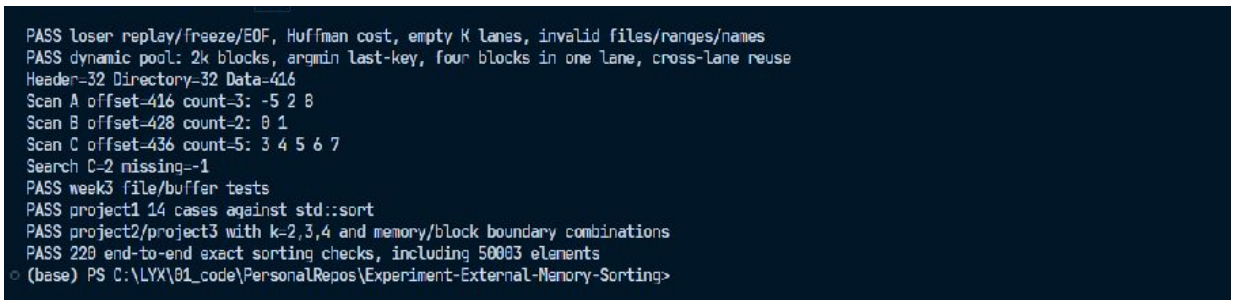


图 4.5: 自动测试：局部机制、文件缓冲与 220 次精确比较通过

4.5 性能指标与 CSV 结果

4.5.1 统计口径

`reads/writes` 是成功的非空数据块 read/write 调用，排除文件头、目录以及演示和验证扫描；不是物理磁盘 I/O。`runs` 指初始段数，`merges` 指实际归并组数，`rounds` 只对 Project 3 计数，Project 1/2 的 0 不代表没有归并。`stages` 累计置换选择与 K 路阶段，

包括启动、等待和排空。

`ms` 用 `steady_clock` 测量 `sortFile`，从建立结果文件之前到归并完成，包括元数据、线程创建、join 及刷新；不包括原始输入生成、参考排序和最终校验。Benchmark 关闭 trace。以下采用已保存的 Windows Release 单次数据，未作均值或冷缓存控制，后台负载和后续重跑可使时间变化。注释后回归的计数与该 CSV 一致，时间不要求相同。

表 4.4: Windows Release 真实 Benchmark，$M = 256, B = 64, k = 4$

项目	N	段数	归并次数	轮数	块读	块写	阶段	ms
1	31	1	0	0	1	1	0	3.7955
2	31	1	0	0	1	1	2	4.74
3	31	1	0	0	1	1	2	3.9383
1	257	2	1	0	10	10	0	2.403
2	257	1	0	0	5	5	6	3.0712
3	257	1	0	0	5	5	6	2.8243
1	4099	17	16	0	386	386	0	4.1211
2	4099	9	8	0	273	278	66	14.3941
3	4099	9	3	2	197	202	198	49.7333
1	50003	196	195	0	6982	6982	0	57.579
2	50003	99	98	0	6106	6152	783	160.114
3	50003	99	34	4	3962	4008	3948	919.6

全部 12 行与 `std::sort` 精确比较均为 PASS；数据源：`benchmark-windows.csv`。

4.5.2 初始段、归并次数与轮数

图 4.6 由同一 CSV 直接生成。$N = 257$ 仅略超过 $M = 256$，普通分块生成 2 段，但该随机序列的置换选择形成 1 段，因此 P2/P3 无需再归并。$N = 50003$ 时段数由 196 降为 99，说明延长段实际减少了后续工作；不能据此推断逆序或所有分布都能减半。

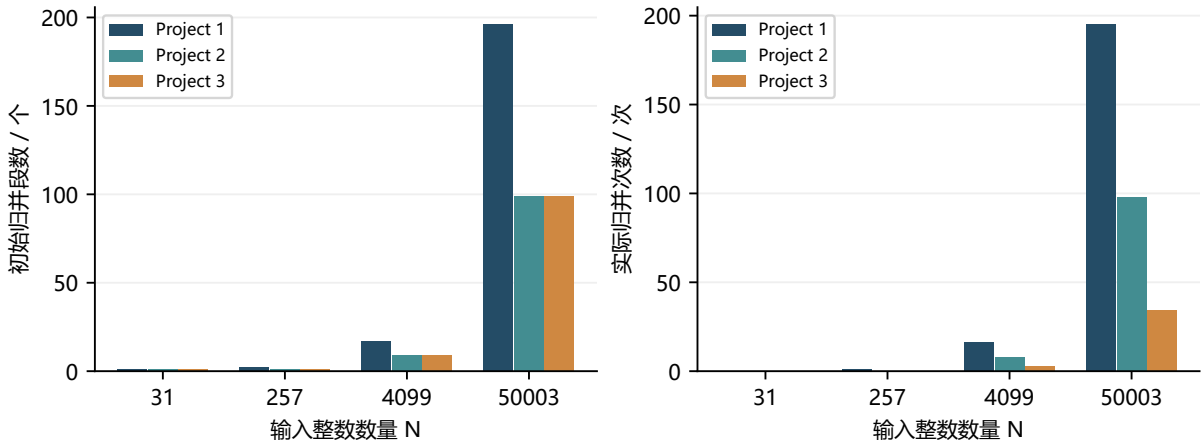


图 4.6: Windows 实测的初始段与归并次数；横轴按四个实验规模分类

P2 对 99 段二路合并 98 次，P3 四路合并 34 次、4 轮。只比较归并次数仍不充分，因为不同组处理的数据量不同，且部分单段直接带过。图 4.7 单独给出 P3 轮数与阶段

数，表明“轮数少”不等于“线程同步少”。

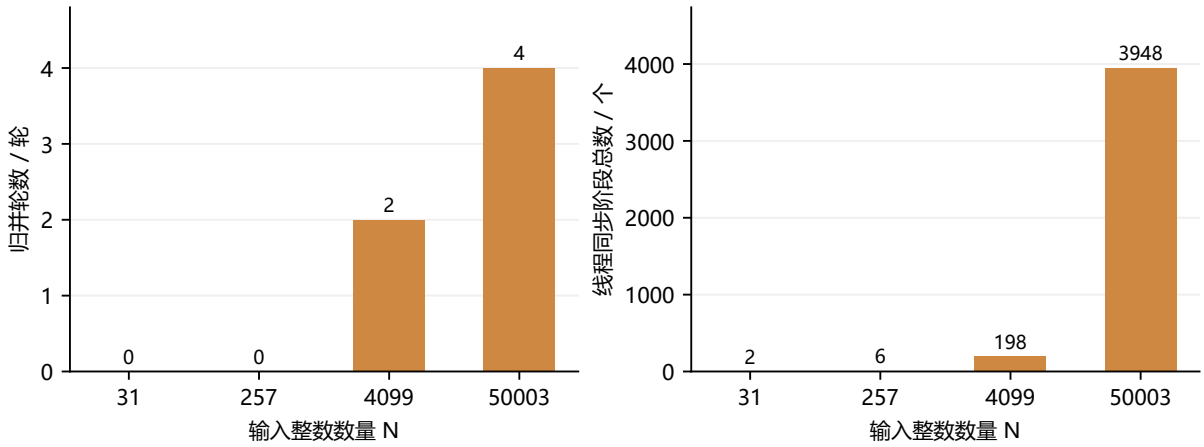


图 4.7: Project 3 的归并轮数与三线程阶段总数，两个纵轴分别标注

默认 15 值下改变 k 也经过实际运行：$k = 2$ 为 2 次归并、2 轮，块读 18、块写 17；$k = 3$ 与 $k = 4$ 都能把 3 个初始段一次合并，均为 1 轮、块读 13、块写 12。这里 $k = 4$ 仍分配 8 个输入块，但本组只有 3 路，说明配置容量与实际参与路数不同。

4.5.3 数据块读写与实际时间

图 4.8 比较最大实验规模的调用数，并展示四种规模的实际排序时间。$N = 50003$ 时 P1 读写合计 13964，P2 为 12258，P3 为 7970。P3 相对 P1 减少约 42.9% 的应用层数据块调用，但计时约为 919.6 ms，高于 P1 的 57.579 ms 和 P2 的 160.114 ms。

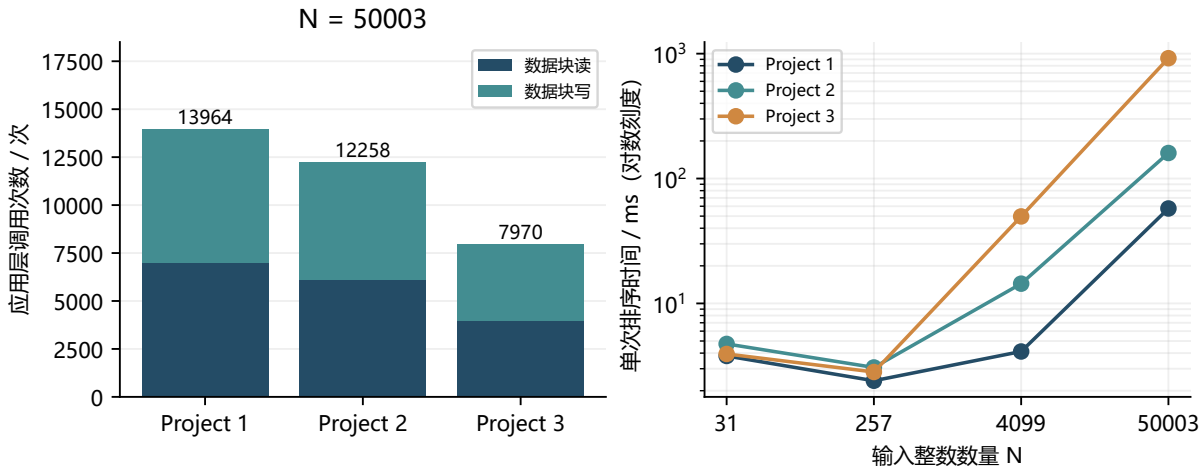


图 4.8: Windows Release 的数据块调用与单次耗时；时间纵轴使用对数刻度

当前 $B = 64$ 仅为 256 字节，一个阶段产生的计算与 I/O 工作很少，而 P3 累计 3948 个同步阶段，每阶段新建并 join 三个线程。测试输入约 200 KB，可被系统缓存，减少逻辑调用并不必然换来相应的物理磁盘节省。线程管理、调度和元数据操作可能占主导。这是对现象的合理解释，实验未单独测量每项开销，不能声称已精确分解各自占比。

`writes` 不总等于 `reads`：置换输出跨段时按段号分拆，K 路缺头时可提前结束阶段并写出不足块的数据。小规模 $N = 31$ 时 P2 和 P3 执行同样的长段生成且无实际 K 路合并，单次时间仍不同，进一步说明不能把微小计时差异直接归因于算法结构。

4.6 Windows 与 WSL 的独立比较及解释边界

表 4.5: $N = 50003$ 时两环境的独立单次计时

环境	Project 1 / ms	Project 2 / ms	Project 3 / ms
Windows Release	57.579	160.114	919.600
此前 WSL Release	21.4028	93.0083	379.418

两份 CSV 除时间外，全部 12 行的段数、归并次数、轮数、块读写与阶段数逐项一致，支持算法行为在两种已测环境中一致。表 4.5 不是操作系统性能排名：编译器、文件系统、缓存与运行时负载未被严格控制，不能把全部差异归于 Windows 或 Linux 本身。

同样 M, B, k 也不代表三项目总内存完全一致。P3 持有更多输入块，置换选择有候选状态与段号标签，目录和计划随段数增长。最大输入 50003 个整数确实远超过候选容量 256，能够触发多轮外排序流程，但尚未超过机器物理内存；不能将它描述为超大规模磁盘压力测试。

本实验支持的结论是：三种实现的正确性及课程机制在覆盖样本中得到验证；长段与 K 路归并减少了相关段数和应用层块调用；当前阶段线程实现并未在此测量条件下获得加速。更大的块、更多重复试验或线程复用可以作为后续研究，不能作为本报告已经完成的优化。

第五章　实验总结

5.1　完成情况与证据链

本实验完成 Week 3–8 的连续工程：底层能够创建、查询和流式追加不等长归并段；Project 1 完成分块内排与后序二路计划；Project 2 完成败者树置换选择、冻结解冻、三线程双缓冲与哈夫曼计划；Project 3 完成共享 $2k$ 输入池、动态预取、真正 K 路选择与多轮主控。

对机制的确认不只依赖最终排序结果。小例子给出段长、树节点、冻结标记和块号变化；独立断言检查一条路占用四块与跨路回收；220 次端到端测试逐项对照完整参考排序；Benchmark 每行也经过精确验证。Windows 的九个具名配置、F5 断点、F10 单步、Ctrl+F5 以及中文路径构建经过实际运行。源码注释最后独立提交，随后再次编译并通过 Debug/Release 完整验收。

5.2　实际遇到的问题与解决过程

5.2.1　构建环境与编码差异

起初 Windows 普通终端没有可调用的 C++ 编译器，VS Code 扩展本身也不提供编译器。先使用已存在的 WSL GCC 完成算法验证，后经用户确认安装微软 Build Tools 的必要 C++ 组件，复用现有 CMake。Windows 使用 Visual Studio 多配置生成器，Debug 和 Release 程序位于各自子目录；不能照搬 Linux 单配置目录中的程序路径。

PowerShell 默认读取 UTF-8 中文材料曾显示乱码，改为显式 UTF-8 读取后恢复。原生 MSVC 编译保留 `/utf-8`，中文脚本采用兼容 Windows PowerShell 的 UTF-8 BOM，文档和源码保持 UTF-8。解决办法是核对读写编码及实际编译日志，而非删除中文注释。此次报告中的源码快照仍保留所有中文说明。

5.2.2　测试目录生命周期与文件系统开销

此前 WSL 把大量微小 I/O 测试放在 Windows 挂载路径，Project 1 阶段测试约 97.7 秒；改到 Linux 用户目录后，同类运行明显加快。因此原始 WSL 性能表明确使用 Linux 文件系统，不与挂载路径数据混合。另一次 WSL 空闲退出后 `/tmp` 构建目录消失，最终改用用户缓存目录，避免临时目录生命周期影响复现。

测试程序使用固定文件名，便于课堂演示和检查目录，但同一目录的多个进程会互相覆盖。当前一键验收顺序执行，Debug、Release 及 CTest 使用明确的工作位置。目录隔离是实验条件的一部分，不应在无意并发运行后把文件冲突解释为归并算法错误。

5.2.3 Ctrl+F5 复用终端的工作目录

实际 GUI 验收发现，C/C++ 扩展 1.35.6 在 Ctrl+F5 复用已有终端时未更新工作目录，可能将相对路径数据写到仓库根。F5 调试子进程的目录正确，但终端 Shell 的提示符仍在原位置，单看提示符不能确认子进程的文件位置。

检查扩展实际行为后，采用最小的 Windows 入口兼容修正：具名启动配置传 `--workdirDIRECTORYMODE`，程序先检查目录并切换，再沿用原模式解析。该修正只放在 Windows 条件编译分支，不改排序接口。中文目录成功、无效目录拒绝、缺参数拒绝均经测试；真实 Ctrl+F5 后运行目录文件更新时间改变，根目录旧文件保持不变。该修正与后续纯注释提交分开保存，避免把行为变化伪装成注释维护。

5.2.4 接口连接、线程收尾和验证口径

先前一次在旧构建仍运行时接入下一阶段源码，CMake 源列表与引用短暂不同步，出现未链接 `replacementSelection`；按完整列表重新配置构建后通过。最终回归期间先停止源码修改，再构建完整工程，避免用更新中的文件混合证明最终状态。

审查中为线程创建失败补充先 join 已启动线程的收尾，防止仍可连接的线程析构触发终止。没有故意制造线程资源耗尽，因此只陈述审查与正常运行验证。尾块、段边界和未知头值分别通过明确状态处理与独立测试覆盖；本报告不虚构曾发生的死锁或丢数事故。

验证口径也作了明确区分。自动生成输入能够完整对照 `std::sort`；用户自定义大文件仅流式检查数量和非降序，并在输出中说明不是精确多重集合比较。保存的 Benchmark 计时不包含参考检查，不将“测试总耗时”当成排序时间，也不把应用层调用称为硬盘物理 I/O。

5.3 学习收获与验收理解

本项目把排序中的抽象“序列”落实为文件中的连续区域。理解一个段不仅需要知道值的顺序，还要知道目录下标、段内元素位置和字节偏移的转换。`beginRun/writeBlock/endRun` 使段长未知时仍能持续输出，缓冲区则让算法保持逐元素逻辑而文件保持分块访问。

归并计划与败者树解决的是两类不同问题。前者安排哪些段先合并，叶子是段；后者决定下一个输出哪个候选，叶子是当前值。哈夫曼计划优化加权搬移量，冠军路径重赛减少重复比较，置换选择通过冻结延长初始段。把它们混成一棵“排序树”会妨碍理解，按接口和生命周期区分后更容易解释复用关系。

线程部分最关键的是数据何时可见、由谁修改。预留块尚未入队，计算不能读取；耗尽块尚未同步，输入不能重用；新输出尚未完成，写线程不能访问。join 不是装饰性等待，而是这些状态交接的边界。实际时间又说明，逻辑上存在并行任务，不意味着在小数据、小块环境下一定更快。

学习和验收时可沿同一 15 值输入追踪三个项目：先定位 R0 的物理偏移，再画普通计划，随后手算冻结与解冻，最后指出物理块 2 怎样跨路使用。报告与中文注释提供可检查依据；对函数的理解仍应通过自己预测下一步状态、修改参数并核对实际结果来巩固。

5.4 局限性与改进方向

1. 文件保留全部初始和中间段，便于观察但增加磁盘占用；尚无空间回收和压缩。
2. 文件结构采用本机字节序，未实现通用序列化、校验和与掉电恢复。输入输出须为不同真实文件，未处理所有路径别名或链接。
3. 候选和数据块受容量约束，目录、段号列表及计划仍为 $O(R)$；并非恒定总内存系统。
4. 每阶段新建三个线程，未实现线程池或复杂异步调度。可考虑复用线程、扩大块并重复测量，但这些不属于已完成结果。
5. 未开展大于物理内存的长期磁盘压力测试，未验证磁盘满、真实掉电、跨字节序及系统线程耗尽。测试不能证明所有未覆盖故障路径。
6. K 路每阶段扫描与重建树、段名线性查询及重复加载目录存在额外成本；后续若优化，应先保持机制与正确性测试，再单独测量收益。

5.5 材料来源与复现文件

课程依据为用户提供的《外存数据管理》85 页课件及两份课堂语音转写；六部分报告结构采用用户提供的 Project Reports 要求。原课件与讲稿仅供本地参考，未发布到公开仓库。本报告的架构图、算法图和说明按最终源码重新绘制和撰写。

实现依据为最终仓库 17 个 C++ 文件、`README.md` 及 `docs/` 中的讲义、课程对应、实验总结与验收问答。Windows 与 WSL 数据分别来自 `benchmark-windows.csv`、`benchmark.csv`；截图为真实 Windows 程序窗口。最后回归原始日志保存在本地 `.local/windows/`，报告目录另外保留必要证据副本。

`report/README.md` 给出 XeLaTeX 编译方法、源码快照生成方法和检查步骤。`scripts/build_report.ps1` 执行编译与验证，`scripts/make_charts.py` 从 CSV 生成矢量图。源代码附录紧接本章并位于整份报告末尾，包含最终实际实现及中文注释。

第六章　完整源代码

本章由脚本直接读取最终仓库文件生成，包含 `main.cpp`、`src/` 和 `tests/` 下全部正式 C++ 头文件与实现文件，不以截图或省略号替代代码。每个文件保留完整中文注释、缩进和行号；较长语句按版心换行，行号仍按源码原始行计算。

快照对应 Git 提交：

`478a7ec6bcc95793a0e5540225609df04986bae2`

正式 C++ 文件共 17 个，另附 CMakeLists.txt 作为构建配置。

快照文件及 SHA-256 摘要保存在 `code_appendix/manifest.json`。`scripts/snapshot_code.py` 检查工作区与提交一致后复制源码，`scripts/verify_report.py` 检查所有快照；普通重新编译不刷新快照，以保证报告的可复现性。

6.1　main.cpp

```
#include "tests/test_sort.h"
#include "src/projects.h"
#include <algorithm>
#include <cerrno>
#include <cstdlib>
#include <iostream>
#include <limits>
#include <random>
#include <stdexcept>
#include <string>
#ifdef _WIN32
#include <direct.h>
#endif
// 严格解析非负整数，避免负数被转换成巨大的无符号容量。
static size_t number(const char* text) {
    if (!text[0]) throw std::runtime_error("empty numeric option");
    for (const char* digit = text; *digit; ++digit)
        if (*digit < '0' || *digit > '9') throw std::runtime_error("numeric option must contain digits only");
    errno = 0;
    char* end = 0;
    const unsigned long long value = std::strtoull(text, &end, 10);
    if (errno || *end || value > std::numeric_limits<size_t>::max()) throw std::runtime_error("numeric option out of range");
    return static_cast<size_t>(value);
}
// 显示统一命令行入口及输入格式；帮助模式不创建数据文件。
static void usage() {
#ifdef _WIN32
    std::cout << "Windows optional prefix: external_sort --workdir DIRECTORY MODE [options]\n";
```

```
#endif
    std::cout << "external_sort week3 | test | benchmark\n"
              << "external_sort project1|project2|project3 [--memory M] [--block B] [--k K] [--trace]\n"
              << "  [--size N | --input raw.bin] [--output runs.bin]\n"
              << "No size/input: 15-value traced demo with exact std::sort verification.\n"
              << "--size: fixed-seed generated data with exact verification.\n"
              << "--input: raw native-endian int32 file; streaming count/order verification.\n"
              << "Output is Header+Directory+Data. M/B are element counts, K >= 2.\n";
}
// 解析模式和容量参数，调用演示、测试或 sortFile。
// 正常退出返回 0；参数、文件或验证错误统一报告并返回 1。
int main(int argc, char** argv) {
    try {
#ifdef _WIN32
        // C/C++ 1.35 的 Ctrl+F5 复用终端时不更新 cwd/环境变量。
        // Windows 验收配置显式传此前缀，保证数据位置；普通命令不受影响。
        if (argc > 1 && std::string(argv[1]) == "--workdir") {
            if (argc < 4) throw std::runtime_error("--workdir needs a directory and mode");
            if (_chdir(argv[2]) != 0) throw std::runtime_error("cannot enter --workdir directory");
            argc -= 2;
            argv += 2; // 跳过目录前缀，让原有模式解析继续使用 argv[1]。
        }
#endif
        const std::string mode = argc > 1 ? argv[1] : "help";
        if (mode == "help" || mode == "--help") { usage(); return 0; }
        if (mode == "week3" || mode == "test" || mode == "benchmark") {
            if (argc != 2) throw std::runtime_error("this mode accepts no options");
            if (mode == "week3") week3Demo();
            else if (mode == "test") runTests();
            else runBenchmarks();
            return 0;
        }
        if (mode != "project1" && mode != "project2" && mode != "project3")
            throw std::runtime_error("unknown mode; run external_sort --help");
        // M、B 以整数个数计；默认 15 个数用于课堂逐步观察。
        size_t memorySize = 4, blockSize = 3, ways = 3, count = 15;
        bool generatedSize = false, trace = false;
        std::string inputPath, outputPath = mode + "_runs.bin";
        for (int index = 2; index < argc; ++index) {
            const std::string option = argv[index];
            if (option == "--trace") { trace = true; continue; }
            if (++index == argc) throw std::runtime_error("missing option value");
            if (option == "--memory") memorySize = number(argv[index]);
            else if (option == "--block") blockSize = number(argv[index]);
            else if (option == "--k") ways = number(argv[index]);
            else if (option == "--size") { count = number(argv[index]); generatedSize = true; }
            else if (option == "--input") inputPath = argv[index];
            else if (option == "--output") outputPath = argv[index];
            else throw std::runtime_error("unknown option " + option);
        }
        if (generatedSize && !inputPath.empty()) throw std::runtime_error("choose --size or --input");
        if (!memorySize || !blockSize || ways < 2) throw std::runtime_error("M/B must be positive and K >=
        ↪  2");
```

```
        // 自动生成数据可保留参考数组；自定义文件仅做流式数量和有序性验证。
        const bool exactCheck = inputPath.empty();
        std::vector<int32_t> expected;
        if (exactCheck) {
            inputPath = mode + "_input.bin";
            if (generatedSize) {
                std::mt19937 random(20261109);
                expected.resize(count);
                for (size_t index = 0; index < count; ++index) expected[index] = int32_t(random() % 20001)
                ↪  - 10000;
            } else {
                expected = {9, 1, 8, 2, 7, 3, 6, 4, 5, 0, -1, 3, 12, 2, 10};
                trace = true;
            }
            if (outputPath == inputPath) throw std::runtime_error("input and output must differ");
            writeRaw(inputPath, expected);
            if (trace && expected.size() <= 100) {
                std::cout << "input:";
                for (int32_t value : expected) std::cout << ' ' << value;
                std::cout << '\n';
            }
        }
        // 正式算法只接收文件路径；生成参考数据与最终验证均在算法计时之外。
        SortResult result = sortFile(mode[7] - '0', inputPath, outputPath, memorySize, blockSize, ways,
        ↪  trace ? &std::cout : 0);
        if (exactCheck) std::sort(expected.begin(), expected.end());
        RunFile file(outputPath, false);
        uint64_t checked = 0;
        int32_t previous = 0;
        if (result.finalRun >= 0) {
            InputBuffer input(file, result.finalRun, blockSize);
            int32_t value;
            if (trace && result.inputCount <= 100) std::cout << "output:";
            while (input.next(value)) {
                if (checked && value < previous) throw std::runtime_error("output not ordered");
                if (exactCheck && (checked >= expected.size() || expected[size_t(checked)] != value))
                    throw std::runtime_error("std::sort exact comparison failed");
                if (trace && result.inputCount <= 100) std::cout << ' ' << value;
                previous = value;
                ++checked;
            }
            if (trace && result.inputCount <= 100) std::cout << '\n';
        }
        // 数量相同且逐项等于 std::sort，才能证明未丢失或重复数据。
        if (checked != result.inputCount) throw std::runtime_error("output count changed");
        std::cout << (exactCheck ? "PASS std::sort exact comparison\n" : "PASS streaming count/order check
        ↪  (not an exact multiset comparison)\n");
        printResult(result, std::cout);
        std::cout << "result_file=" << outputPath << " final_run=" << result.finalRun;
        if (result.finalRun >= 0) std::cout << " name=" << file.run(result.finalRun).name;
        std::cout << '\n';
        return 0;
    } catch (const std::exception& error) {
```

```
        std::cerr << "ERROR: " << error.what() << '\n';
        return 1;
    }
}
```

6.2 src/buffer.cpp

```
#include "buffer.h"
#include <stdexcept>
// 绑定一个已存在的段并分配 B 个整数；构造时校验容量和段号。
InputBuffer::InputBuffer(RunFile& file, int runIndex, size_t blockSize, IoStats* stats)
    : file_(file), runIndex_(runIndex), data_(blockSize), nextOffset_(0), pos_(0), count_(0),
    ↪  stats_(stats) {
    if (blockSize == 0) throw std::runtime_error("block size must be positive");
    file_.run(runIndex);
}
// 向 value 交付一个整数并返回 true；段耗尽返回 false。
// 仅当当前块全部消费后才补块，nextOffset_ 始终是段内元素下标。
bool InputBuffer::next(int32_t& value) {
    if (pos_ == count_) {
        count_ = file_.readBlock(runIndex_, nextOffset_, data_.data(), data_.size(), stats_);
        nextOffset_ += count_;
        pos_ = 0;
        if (count_ == 0) return false;
    }
    value = data_[pos_++];
    return true;
}
// 绑定当前待写段的文件，分配 B 个整数的输出块；beginRun 由调用者负责。
OutputBuffer::OutputBuffer(RunFile& file, size_t blockSize, IoStats* stats)
    : file_(file), data_(blockSize), count_(0), stats_(stats) {
    if (blockSize == 0) throw std::runtime_error("block size must be positive");
}
// 缓存一个整数；满块立即写入，减少逐元素文件调用。
void OutputBuffer::writeValue(int32_t value) {
    data_[count_++] = value;
    if (count_ == data_.size()) flush();
}
// 写出 count_ 个有效整数并清空计数；尾块需在 endRun 前显式调用。
void OutputBuffer::flush() {
    file_.writeBlock(data_.data(), count_, stats_);
    count_ = 0;
}
```

6.3 src/buffer.h

```
#ifndef BUFFER_H
#define BUFFER_H
#include "run_file.h"

// 算法逐个取值；内部只有一个固定容量块。
class InputBuffer {
public:
    InputBuffer(RunFile& file, int runIndex, size_t blockSize, IoStats* stats = 0);
    // 取得下一值；返回 false 表示段末，value 此时不作为新数据。
    bool next(int32_t& value);
private:
    RunFile& file_; // 绑定的段文件，不拥有其生命周期
    int runIndex_; // 当前输入段的目录下标
    std::vector<int32_t> data_; // 容量为 B 的物理整数块
    uint64_t nextOffset_;          // 下一块在段内的元素下标
    size_t pos_;                  // 当前块下一个元素下标
    size_t count_;                // 当前块有效元素数
    IoStats* stats_; // 可选 I/O 计数器；验证读取可传空指针
};
// 逐元素接收排序结果并按块交给 RunFile；与 InputBuffer 共同隔离算法和 I/O。
class OutputBuffer {
public:
    OutputBuffer(RunFile& file, size_t blockSize, IoStats* stats = 0);
    void writeValue(int32_t value);
    void flush();                 // 调用者在 endRun 前显式刷新尾块
private:
    RunFile& file_; // 绑定的段文件，不拥有其生命周期
    std::vector<int32_t> data_; // 容量为 B 的物理整数块
    size_t count_; // 当前输出块已填入的有效整数数
    IoStats* stats_; // 可选 I/O 计数器；验证读取可传空指针
};
#endif
```

6.4 src/kway_merge.cpp

```
#include "kway_merge.h"
#include "loser_tree.h"
#include <algorithm>
#include <exception>
#include <thread>
#include <stdexcept>

// 按配置 k 分配 2k 个 B 容量物理块；逻辑队列数为本组实际段数（可小于 k）。
BufferPool::BufferPool(size_t ways, size_t blockSize, const std::vector<RunInfo>& runs)
    : blocks_(2 * ways), queues_(runs.size()), next_(runs.size(), 0),
      lengths_(runs.size()), lastKeys_(runs.size(), 0) {
```

```
12      if (ways < 2 || runs.size() > ways || !blockSize) throw std::runtime_error("invalid k-way buffers");
13      for (size_t index = 0; index < blocks_.size(); ++index) {
14          blocks_[index].values.resize(blockSize);
15          free_.push(static_cast<int>(index));
16      }
17      for (size_t lane = 0; lane < runs.size(); ++lane) lengths_[lane] = runs[lane].count;
18  }
19  // 每条非空路先填一块，建立可比较的头值；空段不占块。
20  void BufferPool::initialization(RunFile& source, const std::vector<int>& runs, IoStats& stats) {
21      for (size_t lane = 0; lane < runs.size(); ++lane) {
22          if (lengths_[lane] == 0) continue;
23          const int block = reserve();
24          readPrefetch(source, runs[lane], static_cast<int>(lane), block, stats);
25          finishPrefetch(static_cast<int>(lane), block);
26      }
27  }
28  // 只考虑尚未读完的路；空队列优先，否则取 lastKeys 最小路，平局取先者。
29  // 返回路号而非块号；返回-1 表示所有数据已读入，不代表所有数据已输出。
30  int BufferPool::choosePrefetch() const {
31      int target = -1;
32      for (size_t lane = 0; lane < queues_.size(); ++lane) {
33          if (next_[lane] == lengths_[lane]) continue;
34          // 空队列先补足；正常预取取已读未值最小的一路。
35          if (queues_[lane].empty()) return static_cast<int>(lane);
36          if (target < 0 || lastKeys_[lane] < lastKeys_[target]) target = static_cast<int>(lane);
37      }
38      return target;
39  }
40  // 从共享空闲队列取出物理块号；无空闲块返回-1，预取本阶段不启动。
41  int BufferPool::reserve() {
42      if (free_.empty()) return -1;
43      const int block = free_.front();
44      free_.pop();
45      return block;
46  }
47  // 输入线程向预留块读取该路下一段数据；此时不改逻辑队列和 next_。
48  void BufferPool::readPrefetch(RunFile& source, int runIndex, int lane, int block, IoStats& stats) {
49      InputBlock& input = blocks_.at(static_cast<size_t>(block));
50      input.pos = 0;
51      input.count = source.readBlock(runIndex, next_.at(static_cast<size_t>(lane)), input.values.data(),
        ↪  input.values.size(), &stats);
52      if (!input.count) throw std::runtime_error("unexpected empty prefetch");
53  }
54  // join 后发布已读块：入该路队列，推进已读数量，记录块末关键字。
55  void BufferPool::finishPrefetch(int lane, int block) {
56      const InputBlock& input = blocks_[block];
57      queues_[lane].push(block);
58      next_[lane] += input.count;
59      lastKeys_[lane] = input.values[input.count - 1];
60  }
61  // 队列非空表示存在已公开且尚未消费的头值。
62  bool BufferPool::hasValue(int lane) const { return !queues_.at(static_cast<size_t>(lane)).empty(); }
63  // 队列为空且磁盘仍有未读值，表示头值未知，计算必须等待补块。
```

```
64 bool BufferPool::needsData(int lane) const { return !hasValue(lane) && next_[lane] < lengths_[lane]; }
65 // 取路队首块的当前整数；调用前必须确认 hasValue。
66 int32_t BufferPool::front(int lane) const {
67     const InputBlock& input =
       ↪  blocks_.at(static_cast<size_t>(queues_.at(static_cast<size_t>(lane)).front()));
68     return input.values[input.pos];
69 }
70 // 计算线程消费一个值；块耗尽则出队并登记，暂不放回空闲池。
71 void BufferPool::consume(int lane) {
72     const int block = queues_[lane].front();
73     InputBlock& input = blocks_[block];
74     if (++input.pos == input.count) {
75         queues_[lane].pop();
76         released_.push_back(block);
77     }
78 }
79 // 主线程在 join 后归还耗尽块，使它们下一阶段可服务任意一路。
80 void BufferPool::releaseConsumed() {
81     for (size_t index = 0; index < released_.size(); ++index) {
82         blocks_[released_[index]].pos = blocks_[released_[index]].count = 0;
83         free_.push(released_[index]);
84     }
85     released_.clear();
86 }
87 // 只在同步点打印队列、块位置及 lastKeys；复制队列避免破坏原顺序。
88 void BufferPool::print(std::ostream& output) const {
89     output << "pool free=" << free_.size() << '/' << blocks_.size();
90     for (size_t lane = 0; lane < queues_.size(); ++lane) {
91         output << " lane" << lane << "=[";
92         std::queue<int> queue = queues_[lane];
93         while (!queue.empty()) {
94             int block = queue.front(); queue.pop();
95             output << block << ':' << blocks_[block].pos << '/' << blocks_[block].count << ' ';
96         }
97         output << "] loaded=" << next_[lane] << '/' << lengths_[lane] << " last=" << lastKeys_[lane];
98     }
99     output << '\n';
100 }
101
102 // 合并这一组最多 k 个段，输出线程用 file，输入线程用独立 source 文件流。
103 // 合并 runs 中的一组有序段，返回新增结果段号；ways 限定最多路数。
104 // 双输出块交替，输入池共享；独立读写文件流避免并发修改同一 fstream。
105 int kWayMerge(RunFile& file, const std::vector<int>& runs, size_t ways,
106               size_t blockSize, IoStats& stats, std::ostream* trace) {
107     if (runs.empty() || ways < 2 || runs.size() > ways || !blockSize)
108         throw std::runtime_error("invalid k-way merge group");
109     std::vector<RunInfo> infos;
110     uint64_t total = 0;
111     for (size_t lane = 0; lane < runs.size(); ++lane) {
112         infos.push_back(file.run(runs[lane]));
113         total += infos.back().count;
114     }
115     RunFile source(file.path(), false);
```

```
116     BufferPool pool(ways, blockSize, infos);
117     pool.initialization(source, runs, stats);
118     file.beginRun("K" + std::to_string(file.scan().size()));
119     std::vector<int32_t> outputs[2];
120     outputs[0].reserve(blockSize); outputs[1].reserve(blockSize);
121     int activeOutput = 0;
122     uint64_t emitted = 0;
123     while (emitted < total || !outputs[1 - activeOutput].empty()) {
124         const int lane = pool.choosePrefetch();
125         const int prefetchBlock = lane < 0 ? -1 : pool.reserve();
126         const int writeOutput = 1 - activeOutput;
127         outputs[activeOutput].clear();
128         if (trace) {
129             pool.print(*trace);
130             *trace << "prefetch lane=" << lane << " block=" << prefetchBlock
131                    << " computeOutput=" << activeOutput << " writeOutput=" << writeOutput << '\n';
132         }
133         // 每阶段从已公开队首重建 k 叶败者树；缺少未知头值时本阶段只补数据。
134         std::vector<Candidate> heads(runs.size());
135         bool missingHead = false;
136         for (size_t index = 0; index < runs.size(); ++index) {
137             if (pool.needsData(static_cast<int>(index))) missingHead = true;
138             if (pool.hasValue(static_cast<int>(index))) {
139                 heads[index].valid = true;
140                 heads[index].value = pool.front(static_cast<int>(index));
141             }
142         }
143         // 此实现每阶段建树 O(k)，阶段内替换冠军 O(log k)，复杂度需计入两者。
144         LoserTree tree(heads);
145         std::exception_ptr errors[3];
146         std::thread inputThread, computeThread, outputThread;
147         try {
148             // 预取只写被 reserve 隔离的块，尚未入队，计算线程不能看见它。
149             inputThread = std::thread([&]() {
150                 try {
151                     if (prefetchBlock >= 0) pool.readPrefetch(source, runs[lane], lane, prefetchBlock,
                        ↪  stats);
152                 } catch (...) { errors[0] = std::current_exception(); }
153             });
154             // 计算独占活动输出与消费位置；胜出路补上新头后沿败者树路径更新。
155             computeThread = std::thread([&]() {
156                 try {
157                     if (missingHead) return; // 缺少一路的未知头值时，不能从其他路继续输出。
158                     while (outputs[activeOutput].size() < blockSize && tree.winner() >= 0) {
159                         const int winner = tree.winner();
160                         outputs[activeOutput].push_back(tree.top().value);
161                         ++emitted;
162                         pool.consume(winner);
163                         Candidate next;
164                         if (pool.hasValue(winner)) {
165                             next.valid = true;
166                             next.value = pool.front(winner);
167                         }
```

```
                        tree.replaceWinner(next);
                        if (pool.needsData(winner)) break;
                    }
                } catch (...) { errors[1] = std::current_exception(); }
            });
            // 输出线程写上一阶段完成的块，不能读取本阶段正在生成的块。
            outputThread = std::thread([&]() {
                try { file.writeBlock(outputs[writeOutput].data(), outputs[writeOutput].size(), &stats); }
                catch (...) { errors[2] = std::current_exception(); }
            });
        } catch (...) {
            // 若创建后续线程失败，先收回已启动线程，避免析构时 terminate。
            if (inputThread.joinable()) inputThread.join();
            if (computeThread.joinable()) computeThread.join();
            if (outputThread.joinable()) outputThread.join();
            throw;
        }
        // 三个任务结束才回收、公开预取块和切换输出；消除阶段间读写冲突。
        inputThread.join(); computeThread.join(); outputThread.join();
        for (int index = 0; index < 3; ++index) if (errors[index]) std::rethrow_exception(errors[index]);
        ++stats.stages;
        pool.releaseConsumed();
        if (prefetchBlock >= 0) pool.finishPrefetch(lane, prefetchBlock);
        if (trace) *trace << "kway sync " << stats.stages << " emitted=" << emitted << '/' << total
                          << " produced=" << outputs[activeOutput].size() << " free=" << pool.freeCount()
                          ↪  << '\n';
        outputs[writeOutput].clear();
        activeOutput = writeOutput;
    }
    ++stats.merges;
    return file.endRun();
}

// 一轮按组归并，每组是真正的 K 路选择；余下单段直接带入下一轮。
// 反复按至多 k 段分组，返回唯一最终段；空输入返回-1。
// 每轮段数缩为 ceil(R/k)，余下单段直接携带，不执行额外数据复制。
int mergeKRuns(RunFile& file, std::vector<int> runs, size_t ways,
               size_t blockSize, IoStats& stats, std::ostream* trace) {
    if (ways < 2 || !blockSize) throw std::runtime_error("k must be at least two");
    while (runs.size() > 1) {
        ++stats.rounds;
        if (trace) *trace << "merge round " << stats.rounds << " runs=" << runs.size() << '\n';
        std::vector<int> next;
        for (size_t begin = 0; begin < runs.size(); begin += ways) {
            const size_t end = std::min(runs.size(), begin + ways);
            if (end - begin == 1) next.push_back(runs[begin]);
            else {
                std::vector<int> group(runs.begin() + begin, runs.begin() + end);
                next.push_back(kWayMerge(file, group, ways, blockSize, stats, trace));
            }
        }
        runs.swap(next);
    }
```

```
220     return runs.empty() ? -1 : runs[0];
221 }
```

6.5 src/kway_merge.h

```
1  #ifndef KWAY_MERGE_H
2  #define KWAY_MERGE_H
3  #include "run_file.h"
4  #include <queue>
5  #include <ostream>
6  // 共享池的一个物理块：整数数组及有效范围，不固定归属于某一路。
7  struct InputBlock {
8      std::vector<int32_t> values; // 固定 B 容量的数据区
9      size_t count = 0; // 本块有效整数数，尾块可小于 B
10     size_t pos = 0; // 下一个待消费位置
11 };
12 // 固定 2k 个物理块，逻辑上按路排队；块没有永久所属路。
13 class BufferPool {
14 public:
15     BufferPool(size_t ways, size_t blockSize, const std::vector<RunInfo>& runs);
16     // 每个非空段先读一块；其余物理块留在共享空闲池。
17     void initialization(RunFile& source, const std::vector<int>& runs, IoStats& stats);
18     // 返回待预取的路号：优先补空队列，否则选已读末值最小者；无任务返回 -1。
19     int choosePrefetch() const;
20     // 主线程保留一个空闲物理块，未完成读取前不加入计算队列。
21     int reserve();
22     // 输入线程只写被保留的块；同步后 finishPrefetch 才公开它。
23     void readPrefetch(RunFile& source, int runIndex, int lane, int block, IoStats& stats);
24     void finishPrefetch(int lane, int block);
25     bool hasValue(int lane) const;
26     bool needsData(int lane) const;
27     int32_t front(int lane) const;
28     // 计算线程消费一个值，耗尽的块先登记，join 后统一归还。
29     void consume(int lane);
30     void releaseConsumed();
31     void print(std::ostream& output) const;
32     size_t freeCount() const { return free_.size(); }
33     size_t blockCount() const { return blocks_.size(); }
34 private:
35     std::vector<InputBlock> blocks_; // 固定分配，线程运行时不 resize
36     std::vector<std::queue<int> > queues_; // 每路的块编号队列
37     std::queue<int> free_;          // 仅主线程在同步点改动
38     std::vector<int> released_;     // 计算线程记下已耗尽块，同步后归还
39     std::vector<uint64_t> next_;    // 每路已读入的元素数，主线程更新
40     std::vector<uint64_t> lengths_; // 每条输入路的总元素数，用于判断真正读完
41     std::vector<int32_t> lastKeys_; // 每路最后一个已读关键字，预取依据
42 };
43 // 合并一组至多 ways 段并返回结果目录下标；双输出配合三线程阶段同步。
44 int kWayMerge(RunFile& file, const std::vector<int>& runs, size_t ways,
45               size_t blockSize, IoStats& stats, std::ostream* trace = 0);
```

```
// 多轮分组 K 路归并；返回最终段号，空输入返回-1。
int mergeKRuns(RunFile& file, std::vector<int> runs, size_t ways,
               size_t blockSize, IoStats& stats, std::ostream* trace = 0);
#endif
```

6.6 src/loser_tree.cpp

```
#include "loser_tree.h"
#include <stdexcept>
#include <limits>
#include <utility>
// 将候选数补至 2 的幂后建树，额外叶子无效；空输入也保留一个无效叶。
LoserTree::LoserTree(const std::vector<Candidate>& candidates) : leafCount_(1), champion_(0) {
    if (candidates.size() > size_t(std::numeric_limits<int>::max() / 2))
        throw std::runtime_error("too many loser-tree leaves");
    while (size_t(leafCount_) < candidates.size()) leafCount_ *= 2;
    leaves_ = candidates;
    leaves_.resize(leafCount_);
    losers_.resize(leafCount_, -1);
    champion_ = build(1);
}
// 无效值不用 INT_MAX 作哨兵，因而真实 INT_MAX 可以正常参与排序。
// 比较叶下标：有效优先、未冻结优先、值小优先、下标小优先。
bool LoserTree::less(int left, int right) const {
    const Candidate& a = leaves_[left];
    const Candidate& b = leaves_[right];
    if (a.valid != b.valid) return a.valid;
    if (!a.valid) return left < right;
    if (a.frozen != b.frozen) return !a.frozen;
    if (a.value != b.value) return a.value < b.value;
    return left < right;
}
// 递归比赛：返回子树冠军，内部节点只保存败者的叶下标。
int LoserTree::build(int node) {
    if (node >= leafCount_) return node - leafCount_;
    int left = build(node * 2), right = build(node * 2 + 1);
    if (less(right, left)) std::swap(left, right);
    losers_[node] = right;
    return left;
}
// 返回有效冠军的叶下标；所有叶无效时返回-1。
int LoserTree::winner() const { return leaves_[champion_].valid ? champion_ : -1; }
// 查看冠军候选；调用者先用 winner 判断是否仍有有效元素。
const Candidate& LoserTree::top() const { return leaves_[champion_]; }
// 用新候选替换当前冠军，仅沿该叶到根的路径重赛，时间 O(log M)。
void LoserTree::replaceWinner(const Candidate& candidate) {
    if (winner() < 0) throw std::runtime_error("cannot replace an empty tree winner");
    int challenger = champion_;
    leaves_[challenger] = candidate;
    // 原冠军沿途击败的对手恰好保存在这些节点，不需要重算其他子树。
```

```
    for (int node = (leafCount_ + challenger) / 2; node > 0; node /= 2) {
        if (less(losers_[node], challenger)) std::swap(challenger, losers_[node]);
    }
    champion_ = challenger;
}
// 开始新段：清除冻结位并重建比赛；有效性不变，因此不会复活 EOF 叶。
void LoserTree::thaw() {
    for (size_t index = 0; index < leaves_.size(); ++index) leaves_[index].frozen = false;
    champion_ = build(1);
}
// 输出冠军、各内部败者和有效叶；冻结标记用于课堂跟踪段边界。
void LoserTree::print(std::ostream& output) const {
    output << "champion=" << winner() << " losers:";
    for (int node = 1; node < leafCount_; ++node) output << ' ' << node << ':' << losers_[node];
    output << " leaves:";
    for (size_t index = 0; index < leaves_.size(); ++index)
        if (leaves_[index].valid) output << ' ' << index << '=' << leaves_[index].value <<
        ↪  (leaves_[index].frozen ? "(frozen)" : "");
    output << '\n';
}
```

6.7 src/loser_tree.h

```
#ifndef LOSER_TREE_H
#define LOSER_TREE_H
#include <cstdint>
#include <vector>
#include <ostream>
// 败者树叶子：数值与状态分开，完整支持 int32 极值。
struct Candidate {
    int32_t value = 0; // 有效时参与比较的整数
    bool valid = false;            // false 表示输入结束或补齐叶子
    bool frozen = false;           // true 表示只属于下一段
};
// 内部节点只存败者；总冠军单独存放。更新限定为替换当前冠军。
class LoserTree {
public:
    explicit LoserTree(const std::vector<Candidate>& candidates);
    int winner() const;
    const Candidate& top() const;
    // 只替换当前冠军并沿路径重赛；可传无效候选表示该叶输入结束。
    void replaceWinner(const Candidate& candidate);
    // 开始下一段时清除冻结状态并重建树。
    void thaw();
    void print(std::ostream& output) const;
private:
    std::vector<Candidate> leaves_; // 补齐到 2 的幂，额外叶子无效
    std::vector<int> losers_;       // losers_[1..leafCount_-1] 是各场败者的叶下标
    int leafCount_; // 补齐后的叶子数量
```

```
27      int champion_; // 当前总冠军叶下标；有效性由叶状态判断
28      bool less(int left, int right) const;
29      int build(int node);            // 返回该子树冠军，并保存内部败者
30  };
31  #endif
```

6.8 src/merge_plan.cpp

```
1   #include "merge_plan.h"
2   #include <algorithm>
3   #include <stdexcept>
4   #include <queue>
5   #include <functional>
6
7   // memorySize 是本次内排的元素上限；输出继续复用 Week 3 缓冲区。
8   // 从 rawPath 每次取至多 M 个整数内排，按 B 输出；返回初始段目录下标序列。
9   std::vector<int> runGeneration(const std::string& rawPath, RunFile& file,
10                                 size_t memorySize, size_t blockSize, IoStats& stats) {
11      if (!memorySize || !blockSize) throw std::runtime_error("memory and block sizes must be positive");
12      RawInput input(rawPath);
13      std::vector<int32_t> values(memorySize);
14      std::vector<int> runs;
15      while (true) {
16          // 读取调用也受 blockSize 限制，便于同一块大小下比较 I/O。
17          size_t count = 0;
18          while (count < memorySize) {
19              size_t read = input.readBlock(values.data() + count, std::min(blockSize, memorySize - count),
                ↪  &stats);
20              if (!read) break;
21              count += read;
22          }
23          if (!count) break;
24          std::sort(values.begin(), values.begin() + count);
25          file.beginRun("R" + std::to_string(runs.size()));
26          OutputBuffer output(file, blockSize, &stats);
27          for (size_t index = 0; index < count; ++index) output.writeValue(values[index]);
28          output.flush();
29          runs.push_back(file.endRun());
30      }
31      return runs;
32  }
33  // 普通计划：每轮相邻配对；奇数个节点时最后一个直接进入下一轮。
34  // 输入段号对应计划叶子；返回普通二叉计划，空输入 root=-1。
35  MergePlan buildMergePlan(const RunFile& file, const std::vector<int>& runs) {
36      MergePlan plan;
37      std::vector<int> level;
38      for (size_t index = 0; index < runs.size(); ++index) {
39          MergeNode leaf;
40          leaf.runIndex = runs[index];
```

```
        leaf.length = file.run(runs[index]).count;
        level.push_back(static_cast<int>(plan.nodes.size()));
        plan.nodes.push_back(leaf);
    }
    while (level.size() > 1) {
        std::vector<int> next;
        for (size_t index = 0; index < level.size(); index += 2) {
            if (index + 1 == level.size()) { next.push_back(level[index]); continue; }
            MergeNode node;
            node.left = level[index];
            node.right = level[index + 1];
            node.length = plan.nodes[node.left].length + plan.nodes[node.right].length;
            plan.cost += node.length;
            next.push_back(static_cast<int>(plan.nodes.size()));
            plan.nodes.push_back(node);
        }
        level.swap(next);
    }
    if (!level.empty()) plan.root = level[0];
    return plan;
}
// 两路归并只保留两个输入块、一个输出块，以及两个当前值。
// 合并 leftRun/rightRun 两个已排序段，返回新段目录下标。
// 两路相等时先取左路；一方耗尽后顺序排空另一方。
int twoWayMerge(RunFile& file, int leftRun, int rightRun, size_t blockSize, IoStats& stats) {
    InputBuffer left(file, leftRun, blockSize, &stats);
    InputBuffer right(file, rightRun, blockSize, &stats);
    file.beginRun("M" + std::to_string(file.scan().size()));
    OutputBuffer output(file, blockSize, &stats);
    int32_t leftValue = 0, rightValue = 0;
    bool hasLeft = left.next(leftValue), hasRight = right.next(rightValue);
    while (hasLeft || hasRight) {
        if (hasLeft && (!hasRight || leftValue <= rightValue)) {
            output.writeValue(leftValue);
            hasLeft = left.next(leftValue);
        } else {
            output.writeValue(rightValue);
            hasRight = right.next(rightValue);
        }
    }
    output.flush();
    ++stats.merges;
    return file.endRun();
}
// 后序执行：叶子已有数据；内部节点等两个孩子完成后再归并。
// 执行 nodeIndex 子树并缓存结果段号；空树返回-1，已有结果直接复用。
int executeMergePlan(MergePlan& plan, int nodeIndex, RunFile& file,
                     size_t blockSize, IoStats& stats, std::ostream* trace) {
    if (nodeIndex == -1) return -1;
    MergeNode& node = plan.nodes.at(static_cast<size_t>(nodeIndex));
    if (node.runIndex >= 0) return node.runIndex;
    const int left = executeMergePlan(plan, node.left, file, blockSize, stats, trace);
    const int right = executeMergePlan(plan, node.right, file, blockSize, stats, trace);
```

```
94      node.runIndex = twoWayMerge(file, left, right, blockSize, stats);
95      if (trace) *trace << "merge " << file.run(left).name << " + " << file.run(right).name
96                        << " -> " << file.run(node.runIndex).name << " length=" << node.length << '\n';
97      return node.runIndex;
98  }
99  // 打印节点、孩子和理论加权长度 cost，帮助核对计划与执行顺序。
100 void printMergePlan(const MergePlan& plan, std::ostream& output) {
101     for (size_t index = 0; index < plan.nodes.size(); ++index) {
102         const MergeNode& node = plan.nodes[index];
103         output << "node " << index << " length=" << node.length;
104         if (node.left < 0) output << " leaf run=" << node.runIndex;
105         else output << " children=(" << node.left << ',' << node.right << ')';
106         output << '\n';
107     }
108     output << "root=" << plan.root << " theoretical_cost=" << plan.cost << '\n';
109 }
110 // 哈夫曼思想：每次取两个最短段，最小化所有归并输出长度之和。
111 // 以段长为权构造最优二叉归并树；等长时按节点下标确定顺序。
112 // cost 是元素搬移权重，不是含尾块取整和目录访问的实际 I/O 次数。
113 MergePlan buildOptimalPlan(const RunFile& file, const std::vector<int>& runs) {
114     MergePlan plan;
115     typedef std::pair<uint64_t, int> Item;
116     std::priority_queue<Item, std::vector<Item>, std::greater<Item> > queue;
117     for (size_t index = 0; index < runs.size(); ++index) {
118         MergeNode leaf;
119         leaf.runIndex = runs[index];
120         leaf.length = file.run(runs[index]).count;
121         queue.push(Item(leaf.length, static_cast<int>(plan.nodes.size())));
122         plan.nodes.push_back(leaf);
123     }
124     while (queue.size() > 1) {
125         MergeNode node;
126         node.left = queue.top().second; queue.pop();
127         node.right = queue.top().second; queue.pop();
128         node.length = plan.nodes[node.left].length + plan.nodes[node.right].length;
129         plan.cost += node.length;
130         queue.push(Item(node.length, static_cast<int>(plan.nodes.size())));
131         plan.nodes.push_back(node);
132     }
133     if (!queue.empty()) plan.root = queue.top().second;
134     return plan;
135 }
```

6.9 src/merge_plan.h

```
#ifndef MERGE_PLAN_H
#define MERGE_PLAN_H
#include "buffer.h"
#include <ostream>
// 二叉计划节点；保存依赖关系和权重，执行后缓存实际生成的段号。
struct MergeNode {
    int left = -1;                // -1 表示叶子
    int right = -1; // 右孩子节点下标；叶子保持-1
    int runIndex = -1;            // 叶子对应原段；内部节点执行后记录结果
    uint64_t length = 0; // 子树代表的整数总数，建树时相加
};
// 以节点数组保存完整计划，供打印和后序执行复用。
struct MergePlan {
    std::vector<MergeNode> nodes;   // 孩子下标小于父亲，根在最后
    int root = -1;                // 空输入没有根
    uint64_t cost = 0;            // 内部节点 length 之和，即加权外部路径长度
};
// 按 M 分块内排、按 B 写出；返回新建初始段目录下标。
std::vector<int> runGeneration(const std::string& rawPath, RunFile& file,
                               size_t memorySize, size_t blockSize, IoStats& stats);
// 两种建树都只读目录，返回节点数组；不在建树时执行数据归并。
MergePlan buildMergePlan(const RunFile& file, const std::vector<int>& runs);
MergePlan buildOptimalPlan(const RunFile& file, const std::vector<int>& runs);
// 以两个输入块和一个输出块归并，返回新增段号。
int twoWayMerge(RunFile& file, int leftRun, int rightRun, size_t blockSize, IoStats& stats);
// 后序执行指定子树；trace 可选，仅用于演示。
int executeMergePlan(MergePlan& plan, int nodeIndex, RunFile& file,
                     size_t blockSize, IoStats& stats, std::ostream* trace = 0);
void printMergePlan(const MergePlan& plan, std::ostream& output);
#endif
```

6.10 src/projects.cpp

```
#include "projects.h"
#include "replacement.h"
#include "kway_merge.h"
#include <chrono>
#include <limits>
#include <stdexcept>

// 外排序正式入口。只在测试/演示层使用整份数据的 std::sort 参考结果。
// project 选择 1/2/3；rawPath 为原始整数文件，runPath 为新建段文件。
// memorySize 为候选/内排容量，blockSize 为块容量，ways 为 K 路上限；返回指标和最终段号。
SortResult sortFile(int project, const std::string& rawPath, const std::string& runPath,
                    size_t memorySize, size_t blockSize, size_t ways, std::ostream* trace) {
    if (project < 1 || project > 3 || !memorySize || !blockSize || ways < 2 || rawPath == runPath)
```

```
14          throw std::runtime_error("invalid sorting parameters");
15      SortResult result;
16      RawInput input(rawPath);
17      result.inputCount = input.count();
18      // 最坏初始段数不超过 ceil(N/M)；二叉归并最多再增 R-1 段，预留足够目录。
19      const uint64_t maximumRuns = input.count() / memorySize + (input.count() % memorySize != 0);
20      if (maximumRuns > uint64_t((std::numeric_limits<int>::max() - 1) / 2))
21          throw std::runtime_error("too many runs for this directory format");
22      // 计时包括建段文件、生成及归并，排除调用方数据生成与正确性验证。
23      const auto start = std::chrono::steady_clock::now();
24      RunFile::create(runPath, static_cast<uint32_t>(2 * maximumRuns + 1));
25      RunFile file(runPath);
26      std::vector<int> runs;
27      if (project == 1) runs = runGeneration(rawPath, file, memorySize, blockSize, result.io);
28      else runs = replacementSelection(rawPath, file, memorySize, blockSize, result.io, trace);
29      result.initialRuns = runs.size();
30      if (trace) {
31          for (size_t index = 0; index < runs.size(); ++index) {
32              const RunInfo& info = file.run(runs[index]);
33              *trace << "initial " << info.name << " count=" << info.count << " offset=" << info.offset <<
                ↪  '\n';
34              // 只在显式 trace 中显示数据，不纳入排序 I/O 计数。
35              if (input.count() <= 100) {
36                  InputBuffer buffer(file, runs[index], blockSize);
37                  int32_t value;
38                  while (buffer.next(value)) *trace << value << ' ';
39                  *trace << '\n';
40              }
41          }
42      }
43      // 统一计算普通计划的理论 cost；Project 3 只比较该指标，不执行此二叉计划。
44      MergePlan plan = buildMergePlan(file, runs);
45      result.ordinaryCost = plan.cost;
46      if (project == 2) {
47          plan = buildOptimalPlan(file, runs);
48          result.optimalCost = plan.cost;
49          if (trace) *trace << "fixed_chunk_runs=" << maximumRuns << " ordinary_cost=" <<
            ↪  result.ordinaryCost
50                            << " optimal_cost=" << result.optimalCost << '\n';
51      }
52      if (trace && project != 3) printMergePlan(plan, *trace);
53      if (project == 3) result.finalRun = mergeKRuns(file, runs, ways, blockSize, result.io, trace);
54      else result.finalRun = executeMergePlan(plan, plan.root, file, blockSize, result.io, trace);
55      const auto end = std::chrono::steady_clock::now();
56      result.milliseconds = std::chrono::duration<double, std::milli>(end - start).count();
57      return result;
58  }
59  // 汇总元素数、初始段、归并次数、轮数、逻辑数据块 I/O、阶段数和毫秒。
60  void printResult(const SortResult& result, std::ostream& output) {
61      output << "N=" << result.inputCount << " runs=" << result.initialRuns
62             << " merges=" << result.io.merges << " rounds=" << result.io.rounds
63             << " reads=" << result.io.readBlocks << " writes=" << result.io.writeBlocks
64             << " stages=" << result.io.stages << " ms=" << result.milliseconds << '\n';
```

```
}
```

6.11 src/projects.h

```
#ifndef PROJECTS_H
#define PROJECTS_H
#include "merge_plan.h"
// 正式排序返回的元数据和测量结果；不携带整个排序数组。
struct SortResult {
    int finalRun = -1; // 最终段目录下标；空输入保持-1
    uint64_t inputCount = 0; // 原始整数总数 N
    size_t initialRuns = 0; // 生成阶段结束时的段数 R
    uint64_t ordinaryCost = 0; // 普通二叉计划的理论加权长度
    uint64_t optimalCost = 0; // P2 最优二叉计划的理论加权长度
    IoStats io; // 本次排序的数据块和阶段计数
    double milliseconds = 0; // sortFile 内部 steady_clock 测得的毫秒数
};
// 统一外排序入口；创建 runPath，按项目选择生成及归并策略。
SortResult sortFile(int project, const std::string& rawPath, const std::string& runPath,
                    size_t memorySize, size_t blockSize, size_t ways, std::ostream* trace = 0);
void printResult(const SortResult& result, std::ostream& output);
#endif
```

6.12 src/replacement.cpp

```
#include "replacement.h"
#include "loser_tree.h"
#include <algorithm>
#include <exception>
#include <stdexcept>
#include <thread>

// 每个输出元素携带段号：一个块跨越段边界时，输出线程仍能正确更新目录。
struct GeneratedBlock {
    std::vector<int32_t> values;    // 当前块的有效输出，不超过 B 个
    std::vector<size_t> runNumbers; // 与 values 一一对应的生成段号
};
// 置换选择的单个物理输入块；两份交替承担计算输入和预取目标。
struct ReplacementInput {
    std::vector<int32_t> values;    // 固定 B 个整数的物理输入块
    size_t count = 0;              // 有效元素数量
    size_t pos = 0;                // 计算线程下一次消费的位置
};
// 按 runNumbers 把一个输出块切成同段连续片段。
// 跨段时先提交旧段再开启新段；runs 记录已提交段，末段由主流程收尾。
static void writeGeneratedBlock(const GeneratedBlock& block, RunFile& file,
```

```
                                size_t& writtenRun, bool& runOpen,
                                std::vector<int>& runs, IoStats& stats) {
    size_t begin = 0;
    while (begin < block.values.size()) {
        const size_t number = block.runNumbers[begin];
        if (!runOpen || number != writtenRun) {
            if (runOpen) runs.push_back(file.endRun());
            file.beginRun("R" + std::to_string(number));
            writtenRun = number;
            runOpen = true;
        }
        size_t end = begin + 1;
        while (end < block.values.size() && block.runNumbers[end] == number) ++end;
        file.writeBlock(block.values.data() + begin, end - begin, &stats);
        begin = end;
    }
}
// 用 M 个候选执行置换选择，返回生成段号；B 控制输入和输出块容量。
// 每阶段三个线程分别预取、选冠军、写上一块，join 后才交接缓冲区。
std::vector<int> replacementSelection(const std::string& rawPath, RunFile& file,
                                      size_t memorySize, size_t blockSize, IoStats& stats,
                                      std::ostream* trace) {
    if (!memorySize || !blockSize) throw std::runtime_error("invalid replacement memory size");
    RawInput source(rawPath);
    ReplacementInput inputs[2];
    GeneratedBlock outputs[2];
    for (int index = 0; index < 2; ++index) {
        inputs[index].values.resize(blockSize);
        outputs[index].values.reserve(blockSize);
        outputs[index].runNumbers.reserve(blockSize);
    }
    std::vector<Candidate> candidates(memorySize);
    uint64_t loaded = 0;
    size_t filled = 0;
    // 准备阶段把 M 个候选填入树，始终按不超过 B 的块读取。
    while (filled < memorySize) {
        size_t count = source.readBlock(inputs[0].values.data(), std::min(blockSize, memorySize - filled),
        ↪  &stats);
        if (!count) break;
        for (size_t index = 0; index < count; ++index) {
            candidates[filled].value = inputs[0].values[index];
            candidates[filled++].valid = true;
        }
        loaded += count;
    }
    LoserTree tree(candidates);
    std::vector<Candidate>().swap(candidates); // 初始化副本不再占用候选空间
    if (trace) tree.print(*trace);
    inputs[0].count = source.readBlock(inputs[0].values.data(), blockSize, &stats);
    loaded += inputs[0].count;
    // activeInput 供计算消费，另一输入块供预取；输出也采用两份交替。
    int activeInput = 0, activeOutput = 0;
    size_t currentRun = 0, writtenRun = 0;
```

```
74      bool runOpen = false;
75      std::vector<int> runs;
76      // 树空后还需把上一阶段生成的输出写完，不能立即退出。
77      while (tree.winner() >= 0 || !outputs[1 - activeOutput].values.empty()) {
78          const int spareInput = 1 - activeInput;
79          const int writeOutput = 1 - activeOutput;
80          const bool readNeeded = inputs[spareInput].count == 0 && loaded < source.count();
81          // 阶段内只读取这个快照；不会读取输入线程正在更新的 count。
82          const bool noMoreAfterActive = loaded == source.count() && inputs[spareInput].count == 0;
83          outputs[activeOutput].values.clear();
84          outputs[activeOutput].runNumbers.clear();
85          // 每线程只写自己的异常槽；主线程 join 后统一重新抛出。
86          std::exception_ptr errors[3];
87          std::thread inputThread, computeThread, outputThread;
88          try {
89              // 输入线程只接触备用输入块和原始输入流，不改活动块。
90              inputThread = std::thread([&]() {
91                  try {
92                      if (readNeeded) inputs[spareInput].count =
                        ↪  source.readBlock(inputs[spareInput].values.data(), blockSize, &stats);
93                  } catch (...) { errors[0] = std::current_exception(); }
94              });
95              // 计算线程独占败者树、活动输入位置和活动输出，不执行文件 I/O。
96              computeThread = std::thread([&]() {
97                  try {
98                      ReplacementInput& input = inputs[activeInput];
99                      GeneratedBlock& output = outputs[activeOutput];
100                     while (output.values.size() < blockSize && tree.winner() >= 0) {
101                         // 当前块耗尽但后面仍有数据时先停；不能把暂时缺数据当成 EOF。
102                         if (input.pos == input.count && !noMoreAfterActive) break;
103                         // 最小有效候选也被冻结，说明当前段结束；全体解冻后进入下一段。
104                         if (tree.top().frozen) {
105                             ++currentRun;
106                             tree.thaw();
107                             if (trace) *trace << "thaw -> run " << currentRun << '\n';
108                         }
109                         const int32_t value = tree.top().value;
110                         output.values.push_back(value);
111                         output.runNumbers.push_back(currentRun);
112                         Candidate next;
113                         if (input.pos < input.count) {
114                             next.valid = true;
115                             next.value = input.values[input.pos++];
116                             // 新数小于刚输出值会破坏当前段非降序，因此留到下一段。
117                             next.frozen = next.value < value;
118                             if (trace && next.frozen) *trace << "freeze " << next.value << " < last=" <<
                                ↪  value << '\n';
119                         }
120                         tree.replaceWinner(next);
121                         if (trace && source.count() <= 100) tree.print(*trace);
122                     }
123                 } catch (...) { errors[1] = std::current_exception(); }
124             });
```

```
            // 输出线程只写上一阶段的输出块，维护段文件和已提交段号。
            outputThread = std::thread([&]() {
                try { writeGeneratedBlock(outputs[writeOutput], file, writtenRun, runOpen, runs, stats); }
                catch (...) { errors[2] = std::current_exception(); }
            });
        } catch (...) {
            // 若创建后续线程失败，先收回已启动线程，避免析构时 terminate。
            if (inputThread.joinable()) inputThread.join();
            if (computeThread.joinable()) computeThread.join();
            if (outputThread.joinable()) outputThread.join();
            throw;
        }
        // 阶段屏障：所有读写完成后才能更新 loaded、回收旧输出和切换角色。
        inputThread.join(); computeThread.join(); outputThread.join();
        for (int index = 0; index < 3; ++index) if (errors[index]) std::rethrow_exception(errors[index]);
        ++stats.stages;
        if (readNeeded) loaded += inputs[spareInput].count;
        if (trace) *trace << "replacement sync " << stats.stages << " input=" << activeInput
                          << " consumed=" << inputs[activeInput].pos << '/' << inputs[activeInput].count
                          << " output=" << activeOutput << " count=" <<
                          ↪  outputs[activeOutput].values.size() << '\n';
        // 未用完的输入继续保留；只有真正耗尽才交换，避免段边界造成数据丢失。
        if (inputs[activeInput].pos == inputs[activeInput].count) {
            inputs[activeInput].pos = inputs[activeInput].count = 0;
            activeInput = spareInput;
        }
        outputs[writeOutput].values.clear();
        outputs[writeOutput].runNumbers.clear();
        activeOutput = writeOutput;
    }
    if (runOpen) runs.push_back(file.endRun());
    return runs;
}
```

6.13 src/replacement.h

```
#ifndef REPLACEMENT_H
#define REPLACEMENT_H
#include "run_file.h"
#include <ostream>
// M 个候选的败者树置换选择，B 容量双输入/双输出，按阶段运行三个线程。
// 返回初始段目录下标，trace 非空时输出冻结与同步过程。
std::vector<int> replacementSelection(const std::string& rawPath, RunFile& file,
                                      size_t memorySize, size_t blockSize, IoStats& stats,
                                      std::ostream* trace = 0);
#endif
```

6.14 src/run_file.cpp

```cpp
#include "run_file.h"
#include <algorithm>
#include <cstring>
#include <stdexcept>

// 直接保存固定结构，使用本机字节序；本项目面向常见小端 64 位平台。
// 创建并截断 path，预留 maxRuns 个目录槽；预留区使追加数据无需移动已有段。
void RunFile::create(const std::string& path, uint32_t maxRuns) {
    if (maxRuns == 0) throw std::runtime_error("directory capacity must be positive");
    std::ofstream file(path.c_str(), std::ios::binary | std::ios::trunc);
    if (!file) throw std::runtime_error("cannot create " + path);
    FileHeader header = {};
    std::memcpy(header.magic, "EMSORT1", 8);
    header.maxRuns = maxRuns;
    header.directoryOffset = sizeof(FileHeader);
    // 数据起点 = 文件头字节数 + 目录容量 × 单项字节数。
    header.dataOffset = sizeof(FileHeader) + uint64_t(maxRuns) * sizeof(RunInfo);
    file.write(reinterpret_cast<const char*>(&header), sizeof(header));
    RunInfo empty = {};
    for (uint32_t index = 0; index < maxRuns; ++index)
        file.write(reinterpret_cast<const char*>(&empty), sizeof(empty));
    file.flush();
    if (!file) throw std::runtime_error("cannot initialize " + path);
}

// 打开已有段文件并载入目录；writable=false 用于独立只读输入流。
// 校验标识、容量、段连续性和物理文件长度，拒绝截断或越界目录。
RunFile::RunFile(const std::string& path, bool writable)
    : path_(path), endOffset_(0), writing_(false), writable_(writable) {
    std::ios::openmode mode = std::ios::binary | std::ios::in;
    if (writable) mode |= std::ios::out;
    file_.open(path.c_str(), mode);
    if (!file_) throw std::runtime_error("cannot open " + path);
    file_.read(reinterpret_cast<char*>(&header_), sizeof(header_));
    if (!file_ || std::memcmp(header_.magic, "EMSORT1", 8) != 0 ||
        header_.maxRuns == 0 || header_.runCount > header_.maxRuns ||
        header_.directoryOffset != sizeof(FileHeader) ||
        header_.dataOffset != sizeof(FileHeader) + uint64_t(header_.maxRuns) * sizeof(RunInfo))
        throw std::runtime_error("invalid run file header");
    file_.seekg(0, std::ios::end);
    const std::streamoff fileSize = file_.tellg();
    if (fileSize < 0 || uint64_t(fileSize) < header_.dataOffset)
        throw std::runtime_error("truncated directory");
    runs_.resize(header_.runCount);
    file_.seekg(static_cast<std::streamoff>(header_.directoryOffset));
    endOffset_ = header_.dataOffset;
    for (size_t index = 0; index < runs_.size(); ++index) {
        RunInfo& info = runs_[index];
        file_.read(reinterpret_cast<char*>(&info), sizeof(info));
        if (!file_ || !std::memchr(info.name, 0, sizeof(info.name)) ||
```

```
            info.offset != endOffset_ || info.offset > uint64_t(fileSize) ||
            info.count > (uint64_t(fileSize) - info.offset) / sizeof(int32_t))
            throw std::runtime_error("invalid run directory entry");
        endOffset_ = info.offset + info.count * sizeof(int32_t);
    }
}
// 按段名线性查找已提交目录，返回目录下标；未找到返回-1。
int RunFile::search(const std::string& name) const {
    for (size_t index = 0; index < runs_.size(); ++index)
        if (name == runs_[index].name) return static_cast<int>(index);
    return -1;
}
// 检查目录下标并返回元数据引用；不读取段内整数。
const RunInfo& RunFile::run(int index) const {
    if (index < 0 || size_t(index) >= runs_.size()) throw std::runtime_error("run not found");
    return runs_[index];
}
// 开启唯一的待写段，检查容量与重名；此时只修改内存中的 pending_。
void RunFile::beginRun(const std::string& name) {
    if (!writable_ || writing_) throw std::runtime_error("cannot begin run in this state");
    if (runs_.size() >= header_.maxRuns) throw std::runtime_error("directory is full");
    if (name.empty() || name.size() >= sizeof(pending_.name) || name.find('\0') != std::string::npos ||
    ↪  search(name) >= 0)
        throw std::runtime_error("invalid or duplicate run name");
    pending_ = RunInfo();
    std::memcpy(pending_.name, name.c_str(), name.size());
    pending_.offset = endOffset_;
    writing_ = true;
}
// 把 count 个整数追加到当前段，推进字节位置和元素数；空块不计 I/O。
void RunFile::writeBlock(const int32_t* values, size_t count, IoStats* stats) {
    if (!writing_) throw std::runtime_error("beginRun is required");
    if (count == 0) return;
    file_.clear();
    file_.seekp(static_cast<std::streamoff>(endOffset_));
    file_.write(reinterpret_cast<const char*>(values), static_cast<std::streamsize>(count *
    ↪  sizeof(int32_t)));
    if (!file_) throw std::runtime_error("run data write failed");
    pending_.count += count;
    endOffset_ += count * sizeof(int32_t);
    if (stats) ++stats->writeBlocks;
}
// 先写数据，再写目录，最后公布 runCount；不提供掉电事务保证。
// 提交待写段并返回目录下标；调用前必须刷新调用方的输出尾块。
int RunFile::endRun() {
    if (!writing_) throw std::runtime_error("no pending run");
    file_.seekp(static_cast<std::streamoff>(header_.directoryOffset + runs_.size() * sizeof(RunInfo)));
    file_.write(reinterpret_cast<const char*>(&pending_), sizeof(pending_));
    ++header_.runCount;
    file_.seekp(0);
    file_.write(reinterpret_cast<const char*>(&header_), sizeof(header_));
    file_.flush();
    if (!file_) throw std::runtime_error("run metadata write failed");
```

```
    runs_.push_back(pending_);
    writing_ = false;
    return static_cast<int>(runs_.size() - 1);
}
// 小规模整段写入的便捷接口：开启、写数据、提交；正式排序使用分块接口。
int RunFile::put(const std::string& name, const std::vector<int32_t>& values) {
    beginRun(name);
    writeBlock(values.data(), values.size());
    return endRun();
}
// 从 index 段的 position 元素开始，最多读 capacity 个整数到 values。
// 返回实际数量；恰好到段末返回 0，越过段末抛异常，不能读入相邻段。
size_t RunFile::readBlock(int index, uint64_t position, int32_t* values, size_t capacity, IoStats* stats)
    ↪ {
    const RunInfo& info = run(index);
    if (capacity == 0 || position > info.count) throw std::runtime_error("invalid read range");
    const size_t count = static_cast<size_t>(std::min<uint64_t>(capacity, info.count - position));
    if (count == 0) return 0;
    file_.clear();
    // 段内元素下标乘 sizeof(int32_t) 后，才能加到文件字节偏移。
    file_.seekg(static_cast<std::streamoff>(info.offset + position * sizeof(int32_t)));
    file_.read(reinterpret_cast<char*>(values), static_cast<std::streamsize>(count * sizeof(int32_t)));
    if (!file_) throw std::runtime_error("run data read failed");
    if (stats) ++stats->readBlocks;
    return count;
}
// 原始文件没有目录；由文件字节长度计算整数数量并重置读取位置。
RawInput::RawInput(const std::string& path) : count_(0), position_(0) {
    file_.open(path.c_str(), std::ios::binary);
    if (!file_) throw std::runtime_error("cannot open raw input " + path);
    file_.seekg(0, std::ios::end);
    const std::streamoff bytes = file_.tellg();
    if (bytes < 0 || bytes % sizeof(int32_t) != 0) throw std::runtime_error("raw input is not whole int32
    ↪  values");
    count_ = static_cast<uint64_t>(bytes) / sizeof(int32_t);
    file_.seekg(0);
}
// 顺序读取原始输入的下一块，返回实际整数数；尾块允许不足 capacity。
size_t RawInput::readBlock(int32_t* values, size_t capacity, IoStats* stats) {
    if (capacity == 0) throw std::runtime_error("block size must be positive");
    const size_t count = static_cast<size_t>(std::min<uint64_t>(capacity, count_ - position_));
    if (count == 0) return 0;
    file_.read(reinterpret_cast<char*>(values), static_cast<std::streamsize>(count * sizeof(int32_t)));
    if (!file_) throw std::runtime_error("raw read failed");
    position_ += count;
    if (stats) ++stats->readBlocks;
    return count;
}
// 为演示和测试生成原始二进制输入；覆盖 path，不写文件头和目录。
void writeRaw(const std::string& path, const std::vector<int32_t>& values) {
    std::ofstream file(path.c_str(), std::ios::binary | std::ios::trunc);
    if (!file) throw std::runtime_error("cannot create raw file");
```

```
152     if (!values.empty()) file.write(reinterpret_cast<const char*>(values.data()),
        ↪  static_cast<std::streamsize>(values.size() * sizeof(int32_t)));
153     file.flush();
154     if (!file) throw std::runtime_error("raw write failed");
155 }
```

6.15 src/run_file.h

```
1  #ifndef RUN_FILE_H
2  #define RUN_FILE_H
3  #include <cstdint>
4  #include <fstream>
5  #include <string>
6  #include <vector>
7
8  // 只计数据块的实际 read/write 调用；文件头、目录和验证读取不计入排序指标。
9  struct IoStats {
10     uint64_t readBlocks = 0; // 实际非空数据块读取调用次数
11     uint64_t writeBlocks = 0; // 实际非空数据块写入调用次数
12     uint64_t merges = 0; // 生成新段的归并操作次数
13     uint64_t rounds = 0; // K 路分组归并轮数；二叉计划不记此项
14     uint64_t stages = 0; // 置换选择与 K 路归并的三线程同步阶段总数
15 };
16 // 固定 32 字节文件头，记录目录容量和数据起点；结构采用本机字节序。
17 struct FileHeader {
18     char magic[8];                // 文件标识 EMSORT1
19     uint32_t maxRuns;             // 固定目录容量
20     uint32_t runCount;            // 已提交的目录项数量
21     uint64_t directoryOffset;     // 字节偏移
22     uint64_t dataOffset; // 数据区首字节位置
23 };
24 // 固定 48 字节目录项；用名称、字节偏移和元素数描述一个连续归并段。
25 struct RunInfo {
26     char name[32];                // 最多 31 字节，末尾为零
27     uint64_t offset;              // 第一个 int32_t 的文件字节偏移
28     uint64_t count;               // 元素数，不是字节数
29 };
30 static_assert(sizeof(FileHeader) == 32, "FileHeader layout must be 32 bytes");
31 static_assert(sizeof(RunInfo) == 48, "RunInfo layout must be 48 bytes");
32
33 // 顺序追加归并段；同一对象的文件流只允许一个线程访问。
34 class RunFile {
35 public:
36     // 创建文件并预留目录；已有同名文件会被截断。
37     static void create(const std::string& path, uint32_t maxRuns);
38     explicit RunFile(const std::string& path, bool writable = true);
39     const FileHeader& header() const { return header_; }
40     // Scan 直接返回内存目录；只含 endRun 已经提交的段。
41     const std::vector<RunInfo>& scan() const { return runs_; }
```

```
42      int search(const std::string& name) const;
43      const RunInfo& run(int index) const;
44      // 开启一个待写段，随后可多次 writeBlock，最后必须 endRun 提交。
45      void beginRun(const std::string& name);
46      void writeBlock(const int32_t* values, size_t count, IoStats* stats = 0);
47      int endRun();
48      // 一次性写入小段的便捷接口；正式排序应使用流式追加。
49      int put(const std::string& name, const std::vector<int32_t>& values);
50      // 按段内元素位置读取，最多 capacity 个；返回实际数量，0 表示段末。
51      size_t readBlock(int index, uint64_t position, int32_t* values,
52                       size_t capacity, IoStats* stats = 0);
53      const std::string& path() const { return path_; }
54  private:
55      std::string path_; // 文件路径，供 K 路归并另开只读流
56      std::fstream file_;            // seekg 读，seekp 写
57      FileHeader header_; // 当前文件头的内存副本
58      std::vector<RunInfo> runs_;    // 仅目录常驻内存，数据不常驻
59      RunInfo pending_;              // 正在追加的段，endRun 后才可 search
60      uint64_t endOffset_;           // 下一次追加位置
61      bool writing_; // 是否存在尚未提交的待写段
62      bool writable_; // 是否允许追加与提交
63  };
64
65  // 原始输入仅含连续 int32_t，没有 Header/Directory。
66  class RawInput {
67  public:
68      explicit RawInput(const std::string& path);
69      uint64_t count() const { return count_; }
70      size_t readBlock(int32_t* values, size_t capacity, IoStats* stats = 0);
71  private:
72      std::ifstream file_; // 原始输入的独立顺序读流
73      uint64_t count_; // 文件包含的整数总数
74      uint64_t position_; // 已经读取的整数数量
75  };
76  void writeRaw(const std::string& path, const std::vector<int32_t>& values);
77  #endif
```

6.16 tests/test_sort.cpp

```
1   #include "test_sort.h"
2   #include "src/buffer.h"
3   #include "src/projects.h"
4   #include "src/loser_tree.h"
5   #include "src/kway_merge.h"
6   #include <algorithm>
7   #include <limits>
8   #include <random>
9   #include <iostream>
10  #include <stdexcept>
11  #include <cstdio>
```

```
12
13 // 测试断言失败即抛异常，由 main 报告并返回非零，避免只打印失败却误判成功。
14 static void require(bool condition, const char* message) {
15     if (!condition) throw std::runtime_error(message);
16 }
17 // 构造三路相距较大的键值，验证最小末值预取、一路占四块和跨路复用。
18 static void testBufferPool() {
19     RunFile::create("test_pool.bin", 4);
20     RunFile file("test_pool.bin");
21     std::vector<int> runs;
22     runs.push_back(file.put("low", {0, 1, 2, 3}));
23     runs.push_back(file.put("middle", {100, 101}));
24     runs.push_back(file.put("high", {200, 201}));
25     BufferPool pool(3, 1, file.scan());
26     IoStats stats;
27     pool.initialization(file, runs, stats);
28     require(pool.blockCount() == 6 && pool.freeCount() == 3, "2k input pool initialization");
29     for (int repeat = 0; repeat < 3; ++repeat) {
30         require(pool.choosePrefetch() == 0, "smallest last-key policy");
31         int block = pool.reserve();
32         require(block >= 0, "prefetch allocation");
33         pool.readPrefetch(file, runs[0], 0, block, stats);
34         pool.finishPrefetch(0, block);
35     }
36     // lane0 此时拥有 4 个块，证明没有永久固定每路两个。
37     require(pool.freeCount() == 0, "shared pool use");
38     for (int expected = 0; expected < 4; ++expected) {
39         require(pool.front(0) == expected, "queued block ordering");
40         pool.consume(0);
41     }
42     pool.releaseConsumed();
43     require(pool.freeCount() == 4 && !pool.needsData(0), "release all consumed blocks");
44     require(pool.choosePrefetch() == 1, "finished run must not be prefetched");
45     const int recycled = pool.reserve();
46     require(recycled == 0, "released block should be reusable by another lane");
47     pool.readPrefetch(file, runs[1], 1, recycled, stats);
48     pool.finishPrefetch(1, recycled);
49     pool.consume(1);
50     require(pool.front(1) == 101, "cross-lane buffer reuse");
51     std::cout << "PASS dynamic pool: 2k blocks, argmin last-key, four blocks in one lane, cross-lane
       ↪  reuse\n";
52 }
53 // 测试层允许加载参考数组；正式 sortFile 不接收整个输入数组。
54 // 写入 values 后运行指定项目，逐项对照 std::sort；同时检查所有中间段有序。
55 // M/B/k 由调用者组合；返回真实排序指标，验证读取不计入算法 I/O。
56 static SortResult checkSort(const std::vector<int32_t>& values, int project, size_t memorySize, size_t
   ↪  blockSize, size_t ways) {
57     writeRaw("test_input.bin", values);
58     SortResult result = sortFile(project, "test_input.bin", "test_runs.bin", memorySize, blockSize, ways);
59     std::vector<int32_t> expected = values, actual;
60     std::sort(expected.begin(), expected.end());
61     RunFile file("test_runs.bin", false);
62     if (result.finalRun >= 0) {
```

```
        InputBuffer buffer(file, result.finalRun, blockSize);
        int32_t value;
        while (buffer.next(value)) actual.push_back(value);
    }
    require(actual.size() == values.size(), "element count changed");
    require(std::is_sorted(actual.begin(), actual.end()), "output is not ordered");
    require(actual == expected, "output multiset differs from std::sort");
    // 每个中间归并段也必须有序。
    for (size_t index = 0; index < file.scan().size(); ++index) {
        InputBuffer buffer(file, static_cast<int>(index), blockSize);
        int32_t value = 0, previous = 0;
        bool first = true;
        while (buffer.next(value)) {
            require(first || previous <= value, "intermediate run is not ordered");
            previous = value;
            first = false;
        }
    }
    return result;
}
// 独立核对败者重赛/冻结/EOF、最优计划代价、空路及非法文件与目录访问。
static void testMechanisms() {
    std::vector<Candidate> leaves(5);
    const int32_t values[] = {4, 3, 6, 8, 1};
    for (size_t index = 0; index < leaves.size(); ++index) { leaves[index].value = values[index];
    ↪  leaves[index].valid = true; }
    LoserTree tree(leaves);
    require(tree.winner() == 4 && tree.top().value == 1, "loser-tree initial champion");
    Candidate next;
    next.valid = true; next.value = 0; next.frozen = true;
    tree.replaceWinner(next);
    require(tree.top().value == 3, "frozen candidate participated in current run");
    for (int32_t expected : {3, 4, 6, 8}) {
        require(tree.top().value == expected, "loser replay path");
        tree.replaceWinner(Candidate());
    }
    require(tree.top().frozen && tree.top().value == 0, "frozen data was lost");
    tree.thaw();
    require(!tree.top().frozen && tree.top().value == 0, "thaw failed");
    tree.replaceWinner(Candidate());
    require(tree.winner() == -1, "tree EOF failed");

    RunFile::create("test_plan.bin", 16);
    RunFile file("test_plan.bin");
    std::vector<int> runs;
    for (int length : {1, 2, 3, 100}) runs.push_back(file.put("R" + std::to_string(length),
    ↪  std::vector<int32_t>(length, length)));
    require(buildMergePlan(file, runs).cost == 212, "ordinary plan cost");
    require(buildOptimalPlan(file, runs).cost == 115, "Huffman weighted path cost");
    const int empty = file.put("empty", {});
    IoStats stats;
    int result = kWayMerge(file, {empty, runs[0], empty}, 4, 1, stats);
    require(file.run(result).count == 1, "k-way empty lanes");
```

```
    result = kWayMerge(file, {empty, empty}, 4, 3, stats);
    require(file.run(result).count == 0, "all empty merge lanes");
    bool rejected = false;
    try { int32_t value; file.readBlock(runs[0], 2, &value, 1); }
    catch (const std::exception&) { rejected = true; }
    require(rejected, "read beyond run boundary accepted");
    rejected = false;
    try { file.beginRun("R1"); } catch (const std::exception&) { rejected = true; }
    require(rejected, "duplicate name accepted");
    rejected = false;
    try { InputBuffer bad(file, -1, 1); } catch (const std::exception&) { rejected = true; }
    require(rejected, "missing run accepted");
    {
        std::ofstream bad("test_bad.bin", std::ios::binary);
        bad << 'x';
    }
    rejected = false;
    try { RawInput bad("test_bad.bin"); } catch (const std::exception&) { rejected = true; }
    require(rejected, "partial int32 accepted");
    rejected = false;
    try { RunFile bad("test_bad.bin"); } catch (const std::exception&) { rejected = true; }
    require(rejected, "truncated header accepted");
    std::cout << "PASS loser replay/freeze/EOF, Huffman cost, empty K lanes, invalid
    ↪ files/ranges/names\n";
}
// 创建 A/B/C 不等长段，关闭后重开，展示物理偏移、Scan、Search 及尾块输出。
void week3Demo() {
    RunFile::create("week3.bin", 8);
    {
        RunFile file("week3.bin");
        file.put("A", {-5, 2, 8});
        file.put("B", {0, 1});
        file.beginRun("C");
        OutputBuffer output(file, 2);
        for (int32_t value = 3; value < 8; ++value) output.writeValue(value);
        output.flush();
        file.endRun();
    }
    RunFile file("week3.bin");
    std::cout << "Header=" << sizeof(FileHeader) << " Directory=" << file.header().directoryOffset
              << " Data=" << file.header().dataOffset << '\n';
    for (size_t index = 0; index < file.scan().size(); ++index) {
        const RunInfo info = file.run(static_cast<int>(index));
        std::cout << "Scan " << info.name << " offset=" << info.offset << " count=" << info.count << ":";
        InputBuffer input(file, static_cast<int>(index), 2);
        int32_t value;
        while (input.next(value)) std::cout << ' ' << value;
        std::cout << '\n';
    }
    std::cout << "Search C=" << file.search("C") << " missing=" << file.search("missing") << '\n';
    require(file.search("C") == 2 && file.search("missing") == -1, "search failed");
}
// 先验证局部机制，再做 220 次端到端精确排序；固定种子便于复现。
```

```
166 void runTests() {
167     testMechanisms();
168     testBufferPool();
169     week3Demo();
170     RunFile::create("test_file.bin", 2);
171     {
172         RunFile file("test_file.bin");
173         file.put("empty", {});
174         file.beginRun("tail");
175         IoStats stats;
176         OutputBuffer output(file, 3, &stats);
177         for (int32_t value = -5; value < 5; ++value) output.writeValue(value);
178         output.flush();
179         file.endRun();
180         require(stats.writeBlocks == 4, "write block count");
181         bool rejected = false;
182         try { file.beginRun("full"); } catch (const std::exception&) { rejected = true; }
183         require(rejected, "capacity check");
184     }
185     {
186         RunFile file("test_file.bin", false);
187         InputBuffer empty(file, 0, 3);
188         int32_t value;
189         require(!empty.next(value), "empty run");
190         IoStats stats;
191         InputBuffer input(file, 1, 3, &stats);
192         for (int32_t expected = -5; expected < 5; ++expected)
193             require(input.next(value) && value == expected, "buffer round trip");
194         require(!input.next(value) && stats.readBlocks == 4, "tail or count");
195     }
196     std::remove("test_file.bin");
197     std::cout << "PASS week3 file/buffer tests\n";
198     // 边界集覆盖空、单元素、逆序、重复、负数及 int32 最小/最大值。
199     std::vector<std::vector<int32_t> > cases = {{}, {7}, {3, 2, 1}, {-1, -1, -1, 0, -2},
200         {std::numeric_limits<int32_t>::min(), 0, std::numeric_limits<int32_t>::max()}};
201     std::mt19937 random(20261109);
202     for (size_t count : {size_t(17), size_t(257), size_t(4099)}) {
203         std::vector<int32_t> values(count);
204         for (size_t index = 0; index < count; ++index) values[index] = int32_t(random() % 101) - 50;
205         cases.push_back(values);
206         std::sort(values.begin(), values.end());
207         cases.push_back(values);
208         std::reverse(values.begin(), values.end());
209         cases.push_back(values);
210     }
211     // 14 组数据各测 P1/P2 及 P3 的 k=2/3/4，共 70 次。
212     for (size_t index = 0; index < cases.size(); ++index) {
213         checkSort(cases[index], 1, 23, 7, 3);
214         checkSort(cases[index], 2, 23, 7, 3);
215         for (size_t ways : {size_t(2), size_t(3), size_t(4)})
216             checkSort(cases[index], 3, 23, 7, ways);
217     }
218     // M 与 B 的 9 种组合跨 3 个项目，共 27 次，包含 M<B 和单元素容量。
```

```
219     for (size_t memorySize : {size_t(1), size_t(3), size_t(8)})
220         for (size_t blockSize : {size_t(1), size_t(5), size_t(17)})
221             for (int project = 1; project <= 3; ++project)
222                 checkSort({9, 1, 8, 2, 7, 3, 6, 4, 5, 0, -1, 3, 12, 2, 10}, project, memorySize,
                    ↪  blockSize, 3);
223     std::cout << "PASS project1 " << cases.size() << " cases against std::sort\n";
224     std::cout << "PASS project2/project3 with k=2,3,4 and memory/block boundary combinations\n";
225     // 40 个独立随机种子各测 3 项目，共 120 次；加上最后 3 次大输入，总计 220。
226     for (unsigned seed = 0; seed < 40; ++seed) {
227         std::mt19937 randomCase(seed);
228         std::vector<int32_t> values(randomCase() % 100);
229         for (size_t index = 0; index < values.size(); ++index) values[index] = int32_t(randomCase() % 11)
            ↪  - 5;
230         const size_t memorySize = randomCase() % 9 + 1, blockSize = randomCase() % 11 + 1, ways =
            ↪  randomCase() % 6 + 2;
231         for (int project = 1; project <= 3; ++project) checkSort(values, project, memorySize, blockSize,
            ↪  ways);
232     }
233     std::vector<int32_t> large(50003);
234     for (size_t index = 0; index < large.size(); ++index) large[index] = int32_t(random() % 20001) -
        ↪  10000;
235     for (int project = 1; project <= 3; ++project) checkSort(large, project, 256, 64, 4);
236     std::cout << "PASS 220 end-to-end exact sorting checks, including 50003 elements\n";
237 }
238
239 // CSV 时间只覆盖 sortFile；数据生成、std::sort、逐个中间段检查不计时。
240 // 四个 N 各运行三个项目输出 12 行 CSV，M=256、B=64、k=4。
241 // 每个规模重新初始化同一随机种子，各项目使用同一份输入；时间是单次测量。
242 void runBenchmarks() {
243     std::cout << "project,n,memory,block,k,runs,merges,rounds,reads,writes,stages,ms,exact\n";
244     for (size_t count : {size_t(31), size_t(257), size_t(4099), size_t(50003)}) {
245         std::mt19937 random(20261109);
246         std::vector<int32_t> values(count);
247         for (size_t index = 0; index < count; ++index) values[index] = int32_t(random() % 20001) - 10000;
248         for (int project = 1; project <= 3; ++project) {
249             SortResult result = checkSort(values, project, 256, 64, 4);
250             std::cout << project << ',' << count << ",256,64,4," << result.initialRuns << ','
251                       << result.io.merges << ',' << result.io.rounds << ',' << result.io.readBlocks << ','
252                       << result.io.writeBlocks << ',' << result.io.stages << ',' << result.milliseconds <<
                          ↪  ",PASS\n";
253         }
254     }
255 }
```

6.17 tests/test_sort.h

```
#ifndef TEST_SORT_H
#define TEST_SORT_H
// 展示文件布局、目录检索和块缓冲读写。
void week3Demo();
// 运行机制断言及 220 次与 std::sort 完整结果的精确比较。
void runTests();
// 输出真实 12 行 CSV；排序计时不包括测试参考数组的生成与校验。
void runBenchmarks();
#endif
```

6.18 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(ExternalMemorySorting LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
find_package(Threads REQUIRED)
add_executable(external_sort main.cpp src/run_file.cpp src/buffer.cpp
  src/merge_plan.cpp src/projects.cpp src/loser_tree.cpp src/replacement.cpp
  src/kway_merge.cpp tests/test_sort.cpp)
target_include_directories(external_sort PRIVATE ${CMAKE_CURRENT_SOURCE_DIR})
target_link_libraries(external_sort PRIVATE Threads::Threads)
if(MSVC)
  target_compile_options(external_sort PRIVATE /W4 /utf-8)
else()
  target_compile_options(external_sort PRIVATE -Wall -Wextra -Wpedantic)
endif()
enable_testing()
add_test(NAME correctness COMMAND external_sort test)
add_test(NAME cli_help COMMAND external_sort --help)
add_test(NAME reject_zero_block COMMAND external_sort project3 --block 0)
add_test(NAME reject_negative_memory COMMAND external_sort project2 --memory -1)
add_test(NAME reject_bad_k COMMAND external_sort project3 --k 1)
add_test(NAME reject_missing_value COMMAND external_sort project1 --size)
set_tests_properties(reject_zero_block reject_negative_memory reject_bad_k reject_missing_value
  PROPERTIES WILL_FAIL TRUE)
```